Moderated Learning Management System

A software engineering project report: from planning to maintenance (release v1.1.0)

[Student name and number] · [Course code and title] · Repository: [link to the Git repository]

[Date of submission]

Abstract

This report describes the development of a Moderated Learning Management System (LMS) for a secondary school, taken through the full software life cycle with a waterfall process and three phase gates. The school needs to manage courses, assignments and assessment for students, lecturers, heads of department and an administrator, with one central rule: students must not see a mark until the head of department has checked and released it. The report covers planning and risk management, requirements engineering with conflict resolution between five stakeholders, a layered design with four design patterns and a third-normal-form database, and a C++17 implementation using SQLite, salted Argon2id password hashes and an append-only audit log. Verification used 104 automated tests (98.6% line coverage of the business-logic layer), static analysis and system tests derived from the use cases. Release v1.0.0 was followed by a change request (class-wide extensions) delivered as v1.1.0 with a full regression run. The report ends with a discussion of data protection, accessibility and the fairness of the late-penalty rules, and states plainly what has not yet been verified.

Contents

2 Introduction **4**

3 Project Management Plan **4**

3.1 Feasibility study 4

3.2 Work breakdown structure and schedule 6

3.3 Risk management 7

4 Requirements Engineering **7**

4.1 Stakeholder analysis and conflict resolution 7

4.2 Software requirements specification 8

4.3 Use case model 9

5 Software Design Document **10**

5.1 Architecture 10

5.2 Class design and design patterns 11

5.2.1 Services, patterns and repositories 13

5.3 Behavioural models 15

5.4 Database design 19

6 Implementation **20**

6.1 Structure and key decisions 20

6.2 Security 20

6.3 Persistence 21

6.4 Fidelity to the design 21

7 Verification and Validation **21**

7.1 Test strategy 21

7.2 Automated tests and coverage 21

7.3 System and acceptance testing 22

7.4 Performance and defects 22

7.5 Limits of the evidence 23

8 Configuration Management and Quality Assurance **23**

8.1 Configuration management 23

8.2 Quality assurance 23

9 Deployment **24**

10 Maintenance: Change Request CR1 ..... **24**

10.1 Classification and impact analysis 24

10.2 Design and implementation 26

10.3 Regression testing 27

11 Professional, Legal and Ethical Issues **28**

11.1 Data protection 28

11.2 Security of assessment records 28

11.3 Accessibility..... 28
11.4 Fairness of the late-penalty rules..... 28
11.5 Professional responsibility..... 29
12 Conclusion..... 29
13 References..... 29
14 Appendices..... 30
Appendix A: Requirements traceability matrix..... 31
Appendix B: Full automated test results (v1.1.0)..... 35
Appendix C: User manuals (version 1.1)..... 38
Appendix D: Updated risk register..... 40
Appendix E: Data dictionary and SQL schema..... 41

2 Introduction

A secondary school wants one system to manage courses, assignments and assessment. It has four kinds of user: students, lecturers, heads of department (HoDs) and a system administrator. Lecturers set and mark work, students submit it, and the HoD checks the marks of every course in the department before they are released. Every change to a mark must be recorded, so that the school can see who changed it, when and why. The system runs on one school computer or a local network, must work without the internet, and must be simple enough for teachers who are not IT experts.

The aim of this project was to take that system through the whole software engineering life cycle, making and justifying an engineering decision at each stage: planning and risk management, requirements engineering, architecture and detailed design, implementation, verification and validation, configuration management and quality assurance, deployment, and maintenance after release. The project follows the waterfall model (Royce, 1970; Sommerville, 2016) with a documented deliverable at the end of each phase and three review gates, and implementation was not started until the design had passed its gate. Online payment, video conferencing, automated grading, plagiarism detection and web or mobile interfaces were out of scope.

The main results are as follows. The requirements specification contains 24 functional and 12 non-functional requirements, each prioritised, identified and verifiable. The system is a C++17 console application of about 2,300 lines of code in a layered architecture, with SQLite storage accessed only through a data-access layer. It has 104 automated tests, all passing, and the line coverage of the business-logic layer is 98.6%. A login takes about 80 ms with 800 students stored, against a target of 2 seconds. Release v1.0.0 was followed by change request CR1, which added class-wide deadline extensions and was released as v1.1.0 after all 93 existing tests were re-run unchanged. One non-functional requirement (usability with real lecturers) has not been verified, because it needs real users; this and the other limits of the evidence are stated in section 7.

Section 3 presents the project management plan, section 4 the requirements, section 5 the design, and sections 6 to 10 the implementation, testing, quality assurance, deployment and maintenance. Section 11 discusses the professional, legal and ethical issues, and section 12 concludes. The appendices hold the traceability matrix, the full test results, the user manuals, the updated risk register and the data dictionary with the SQL schema.

3 Project Management Plan

3.1 Feasibility study

Feasibility was assessed on five dimensions, and the recommendation was to proceed under three conditions.

Technical feasibility. C++17, SQLite, CMake, GoogleTest and libsodium are mature, free, and run offline on modest hardware. SQLite is an embedded database that needs no server, and 800 students produce only tens of thousands of rows, so the response-time targets were realistic. The main technical risk was the developer’s own experience with C++ and the tooling, not the technology.

Operational feasibility. The principal requirement that the system be simple points to a menu-driven console with a short menu for each role, which also runs on old computers. A graphical interface would be friendlier but would cost time, so it was classed as out of scope for version 1.0. One manual per role supports adoption.

Economic feasibility. Every dependency is free (SQLite is public domain, libsodium is ISC-licensed and GoogleTest is BSD-licensed), and the system runs on an existing computer. The main cost is development effort, planned at about 245 hours over seven weeks, with low running costs.

Legal feasibility. The system holds personal data about learners and their assessment results, so data protection law applies. The design therefore collects only the data needed, restricts access by role, hashes passwords and keeps an audit log (section 11).

Schedule feasibility. The seven-week deadline is fixed and the phases overlap in weeks 2, 4 and 5, so schedule was rated as the weakest dimension. The mitigation was MoSCoW prioritisation, which allowed the one “Could have” requirement to be dropped without harming the rest.

The recommendation was to proceed, using a console interface in version 1.0, holding the scope to the Must and Should requirements, and re-scoring the risk register at every gate.

3.2 Work breakdown structure and schedule

The work breakdown structure has nine branches that follow the life cycle: planning, requirements, design, implementation, verification and validation, configuration management and quality assurance, deployment, maintenance, and completion. Each branch holds two to four work packages, for example 3.2 “class diagram, patterns and SOLID rationale” and 4.2 “data-access layer, schema and password hashing”. Figure 1 shows the Gantt chart for the 27 tasks, with finish-to-start dependencies, three phase gates as milestones, and the critical path in red.

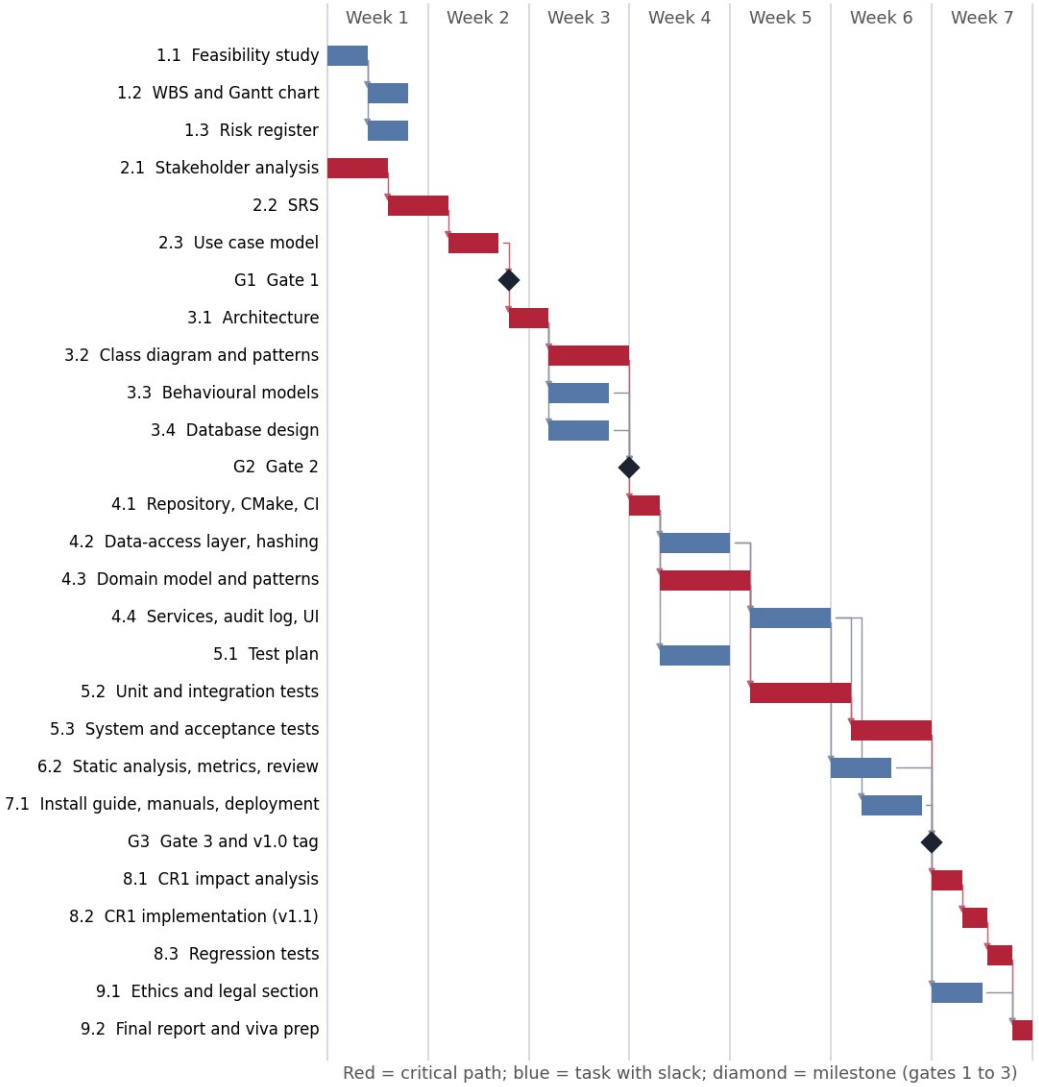


Figure 1: Gantt chart with dependencies, milestones and critical path

The critical path runs from the stakeholder analysis through the SRS, use case model and Gate 1, then architecture, class design and Gate 2, then the repository set-up, the domain model, the unit and integration tests, the system tests and Gate 3 with the v1.0 tag. It continues through the change request analysis, implementation and regression tests to the final report. The data-access layer, the services and user interface, the test plan, the quality checks and the documentation have slack of 0.1 to 0.7 weeks. The durations are planning estimates; the task dates were set when the plan was baselined and were used to order the work, not recorded as actuals.

The plan also serves as a control. The gates are the control points: the plan prescribes the SRS and use case model at Gate 1, the design document at Gate 2, and the test results and the tagged release at Gate 3, and no implementation may start before Gate 2. Because the plan has almost no slack at the gates, the

one “Could have” requirement (the CSV export) was the first thing to be given up rather than letting a Must requirement slip. The durations are estimates and no timesheet is claimed, so this report does not state actual effort; the final risk register is the evidence of how the plan held up.

3.3 Risk management

The register started with eight risks of three kinds (technical, project and people), each with a likelihood and impact from 1 to 5, an exposure (their product), a mitigation and an owner. Table 1 lists the highest-ranked ones; the register was re-scored at each gate and its final form, with three risks discovered during the work, is in Appendix D.

Table 1: Highest-ranked initial risks

ID	Risk	Type	Exposure	Mitigation
R1	Schedule overrun with a fixed seven weeks	Project	20	MoSCoW; Must items first; weekly check
R2	Stakeholder conflicts resolved inconsistently	People	16	Written resolutions baselined at Gate 1
R3	Skill gaps in C++, SQLite and CI	Technical	12	Small interfaces; early tests
R4	Database lost or corrupted	Technical	10	Transactions; backup function; restore guide

At completion, five of the eleven risks were closed and six remained open with a residual exposure of 6 to 9. The open ones are the missing usability test with real lecturers, the unverified build on the school’s old computers, the limits of SQLite on a network folder, the untested native Windows build, and two accepted limitations.

4 Requirements Engineering

4.1 Stakeholder analysis and conflict resolution

Eight stakeholders were identified: the Principal, lecturers, HoDs, students, the student representative, the administrator, the IT officer, and the indirect stakeholders (parents and the data protection regulator). The five interview statements were incomplete and in places contradictory, so eleven conflicts and ambiguities were listed and each was resolved in writing (Table 2).

Table 2: Conflicts and ambiguities in the stakeholder statements, and their resolutions

#	Issue	Resolution
C1	Lecturer wants marks seen at once; Principal and HoD want them checked first	A student sees a mark only when it is Released; lecturers mark immediately and the HoD releases a whole assignment at once
C2	Does the penalty count from the original or extended deadline?	From the student’s effective deadline (extension if any)
C3	“10% of the available mark per day”: of what, and what is a day?	10% of the maximum mark for each started 24 hours, up to 3 periods; later gives zero
C4	Late work must not be rejected, but is zero after three days	Always accepted and flagged Late; feedback still given
C5	May a student resubmit after the deadline?	Only before the effective deadline; late work cannot be replaced
C6	Marking early and then a resubmission	Marking opens after the effective deadline
C7	Is written feedback mandatory?	Yes, to mark a submission

#	Issue	Resolution
C8	“Simple” and “fast” are not measurable	Measurable targets: first-time lecturer creates an assignment in 5 minutes; login in 2 seconds
C9	“800 students on it”: stored or simultaneous?	800 stored; up to 5 simultaneous sessions
C10	Who may change a mark and when?	Lecturer before approval, HoD before and after release; always with a reason and an audit entry
C11	Can one person hold two roles?	No: exactly one role per account

The resolutions of C1 and C3 matter most. For C1 the Principal sponsors the system and HoD moderation is mandatory in the case study, so the HoD’s rule wins, but batch release recovers most of the speed the lecturer wanted. For C3 a fixed, started-day rule is predictable and testable, and avoids a loophole in which work 23 hours late would be free. Several resolutions are assumptions that the stakeholders should confirm, and they are listed in the SRS for Gate 1.

4.2 Software requirements specification

The SRS follows ISO/IEC/IEEE 29148 (2018): introduction, overall description, business rules, and specific requirements, each with a unique identifier, a MoSCoW priority, a source and a verification method. Version 1.0 has 23 functional requirements (18 Must, 4 Should and 1 Could) and 12 non-functional requirements (8 Must and 4 Should). Change request CR1 added FR-24. The ten business rules (BR-1 to BR-10) define the effective deadline, the days-late calculation, the penalty, the five-state submission life cycle, version retention, resubmission, the marking window, roles and the reason rule. For example, BR-3 states that a submission 1 to 3 started days late loses 10% of the maximum mark for each day, and that later work receives zero.

The non-functional requirements are measurable. Examples are NFR-01 (login within 2 seconds with 800 students and 60 staff stored), NFR-04 (passwords only as salted Argon2id hashes), NFR-06 (at least 50 invalid inputs cause no crash), NFR-07 (every mark change produces an audit entry that cannot be edited), and NFR-11 (at least 70% line coverage of the business logic). Each is verified by a test, an inspection, a measurement or a demonstration, as shown in the traceability matrix in Appendix A.

Requirements were drawn from the five interview statements and the case study, and each one records its source (Principal, lecturer, HoD, student representative, IT officer or the case study) so that it can be traced back to a person or document. MoSCoW priorities followed a simple rule. Anything needed for the central moderation rule, or stated in the case study, is a Must. Things that make operation safer or easier, such as password change with lockout, audit viewing, backup and the correction of released marks, are Should. The CSV export is Could. The Won’t-have list repeats the case study’s exclusions. A verification method (test, inspection, demonstration or measurement) was chosen when each requirement was written, because a requirement that cannot be verified cannot be accepted.

4.3 Use case model

The use case model has 21 use cases for the four actors, which specialise an abstract *User* actor (Figure 2). It uses *include* (marking includes applying the late penalty and recording an audit entry), *extend* (resubmission and late submission extend “Submit assignment”, and adjusting a mark extends moderation) and actor generalisation.

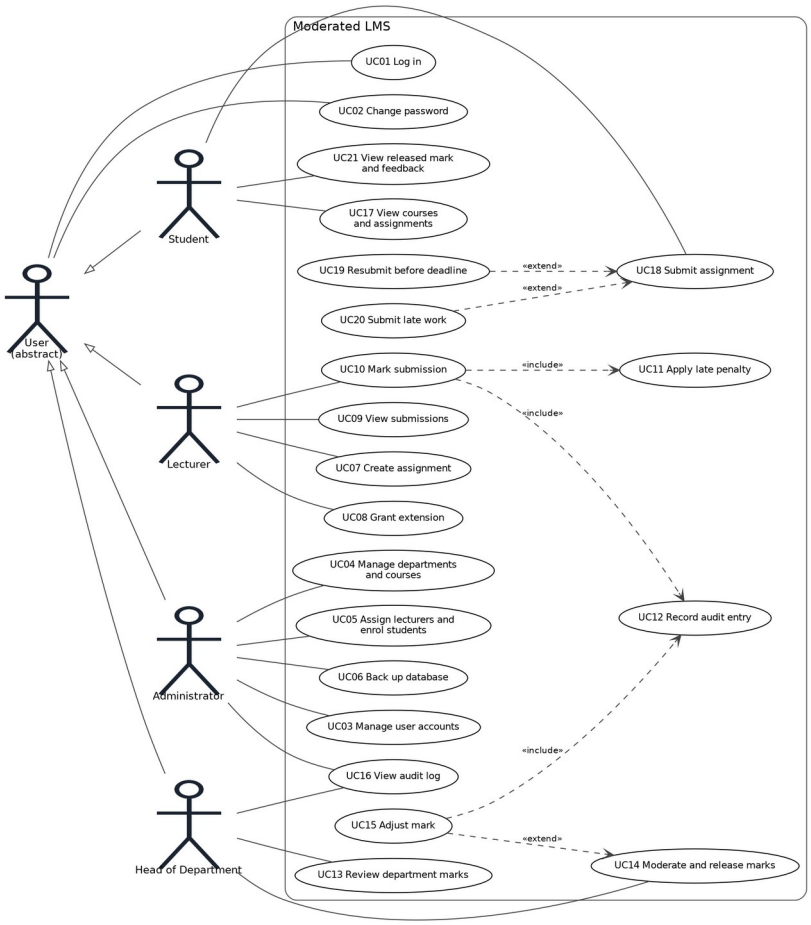


Figure 2: Use case diagram

Three use cases were written as fully dressed specifications with preconditions, main flow, alternative and exception flows, postconditions and business rules: *Submit assignment* (including the resubmission and late paths), *Mark submission* and *Moderate and release marks*. Together they have 31 flows, of which each has a system test (section 7). CR1 later added a twenty-second use case, *Grant class extension*, which extends *Grant extension*.

In *Submit assignment* the student chooses an assignment and a file. The system validates the file (it exists, is readable, is not empty and is at most 5 MB), works out the effective deadline from the assignment and any extension, and compares it with the current time. On time, it stores the file as a new version in state Submitted. If a submission already exists and is still Submitted, the new version becomes the one that will be marked and the earlier version is kept. After the deadline, with no earlier submission, it stores version 1 in state Late and tells the student how many days late the work is. The exception flows cover a student who is not enrolled, an invalid file, an attempt to replace late work or work that has been marked, and a storage error, which must leave nothing half-saved.

5 Software Design Document

5.1 Architecture

The system uses a layered architecture packaged as one program (Figure 3). The presentation layer is a console with a menu per role; the application layer holds one service per group of use cases; the domain layer holds the entities, the role permissions, the penalty rule and the mark-change events; and the data-access layer holds repository interfaces and their SQLite implementations. Dependencies point downward only, and no SQL exists outside the data-access layer.

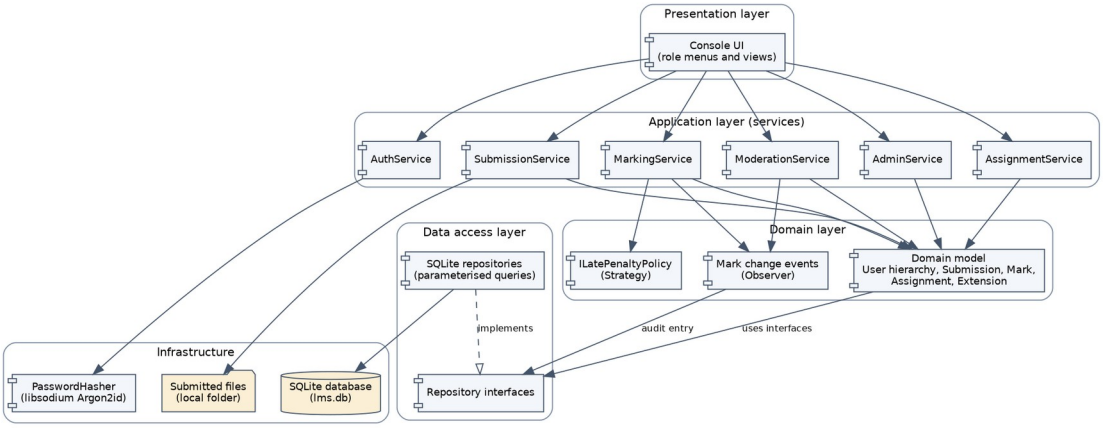


Figure 3: Component diagram

A layered style was chosen because it fits the requirements. A single process with an embedded database satisfies the offline requirement (NFR-03). Services and the domain depend only on interfaces, so tests can replace SQLite by in-memory fakes and the clock by a fixed time, which makes the deadline boundary tests and the coverage target achievable (NFR-11). Password hashing and SQL each live in exactly one place, which makes them easy to inspect (NFR-04, NFR-05). Model-View-Controller was considered and rejected: it separates input from display, which helps event-driven graphical interfaces, but a menu-driven console has no events to route and MVC says nothing about where persistence and business rules belong. A client-server design with a database server was also rejected, because the IT officer requires offline operation on old computers.

Several decisions cut across the layers. Errors use one exception hierarchy, caught only by the console loop. Multi-step changes use a unit of work that rolls back unless it is committed. A `Clock` interface supplies the current time. A SQLite busy timeout with immediate write transactions lets up to five sessions share one database file. Role checks use `User::allows`, so a new role needs a new class and not a change to every service.

5.2 Class design and design patterns

The domain model (Figure 4) has an abstract `User` class with four subclasses. Each subclass implements `allows(Action)`, so role checks need no `if (role == ...)` code. A `Submission` owns its versions and at most one `Mark`. Visibility, types, multiplicities and inheritance are shown on the diagram.

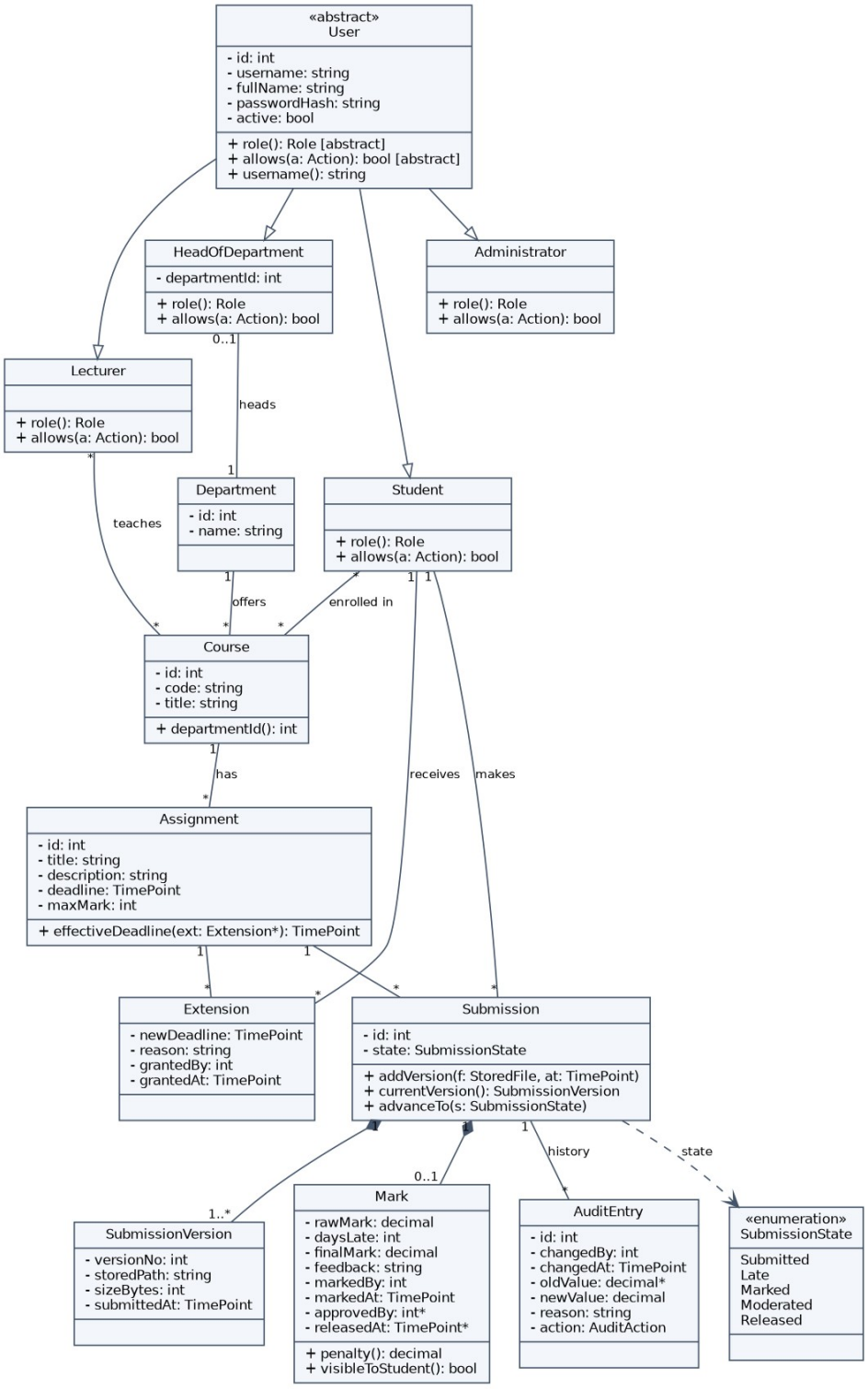


Figure 4: Domain class diagram

Four design patterns are used, each for a stated reason (Gamma et al., 1994; Fowler, 2002).

- **Strategy** for the late-penalty rule. `ILatePenaltyPolicy` has one implementation, `TenPercentPerDayPolicy`. The rule is the part most likely to change, and a new rule needs no change to the marking code. Tests also use it with different parameters to show that it can be replaced.
- **Repository** for data access. Each aggregate has an interface and a SQLite class, which keeps SQL in one layer and lets the services be tested with fakes.
- **Factory** for user creation. `UserFactory` builds the right subclass from a stored role and enforces that a head of department has a department.
- **Observer** for the audit log. Services publish a mark-changed event and `AuditObserver` writes the audit entry inside the same transaction, so no code path can change a mark and forget the audit.

The design follows the SOLID principles (Martin, 2003). Each service has one responsibility; new penalty rules, observers and roles are added as new classes (open-closed); every `User` subclass can replace the base class (Liskov); repository interfaces are small (interface segregation); and services receive interfaces through their constructors, with only `main` creating the concrete SQLite and libsodium classes (dependency inversion).

The late-penalty rule shows how the Strategy pattern works in practice. The policy receives the maximum mark, the raw mark, the effective deadline and the submission time. Days late is zero if the submission is not after the deadline, and otherwise the number of started 24-hour periods. One to three days lose 10% of the maximum mark per day, the result is never below zero, and more than three days gives zero. For a maximum of 20 and a raw mark of 16, work exactly at the deadline keeps 16; one second late gets 14; exactly 24 hours late also gets 14; 24 hours and one second late gets 12; exactly 72 hours late gets 10; and 72 hours and one second late gets 0. These cases became the boundary tests.

5.2.1 Services, patterns and repositories

Figure 5 shows the services with the Strategy and Observer patterns, and Figure 6 shows the Factory, the password hasher and the Repository pattern.

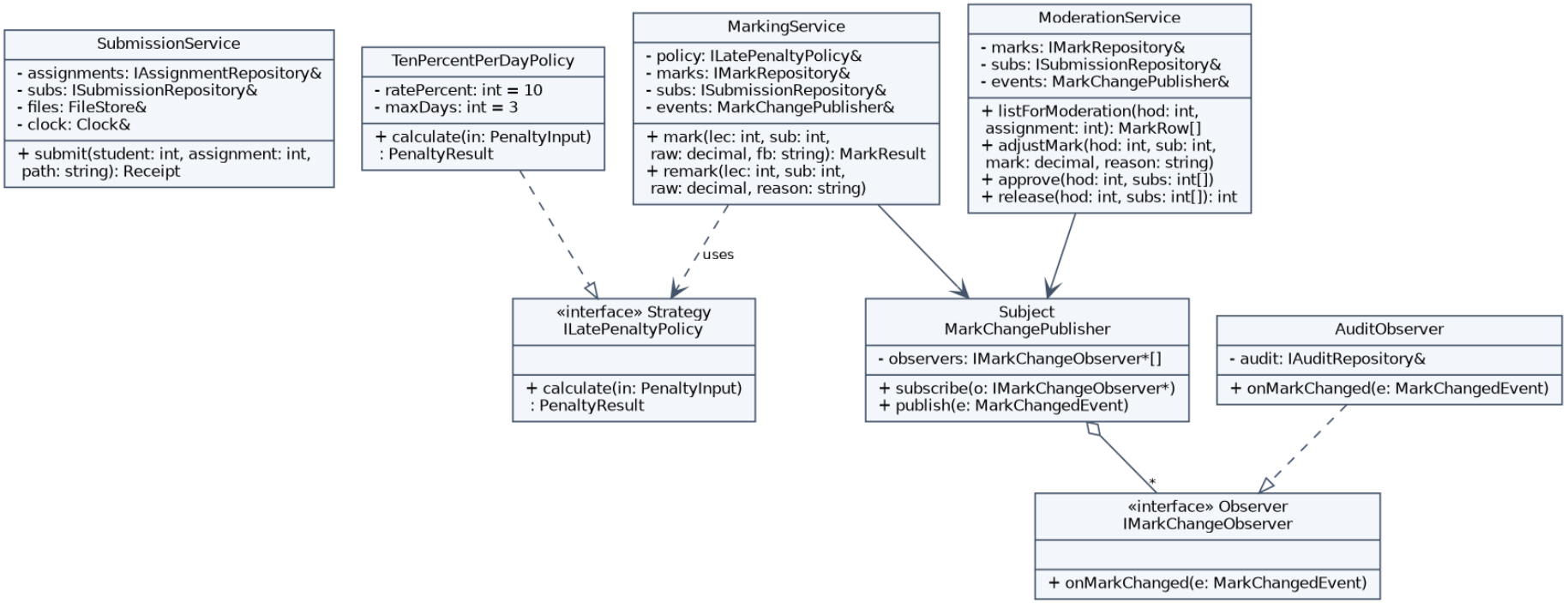


Figure 5: Application services with the Strategy and Observer patterns

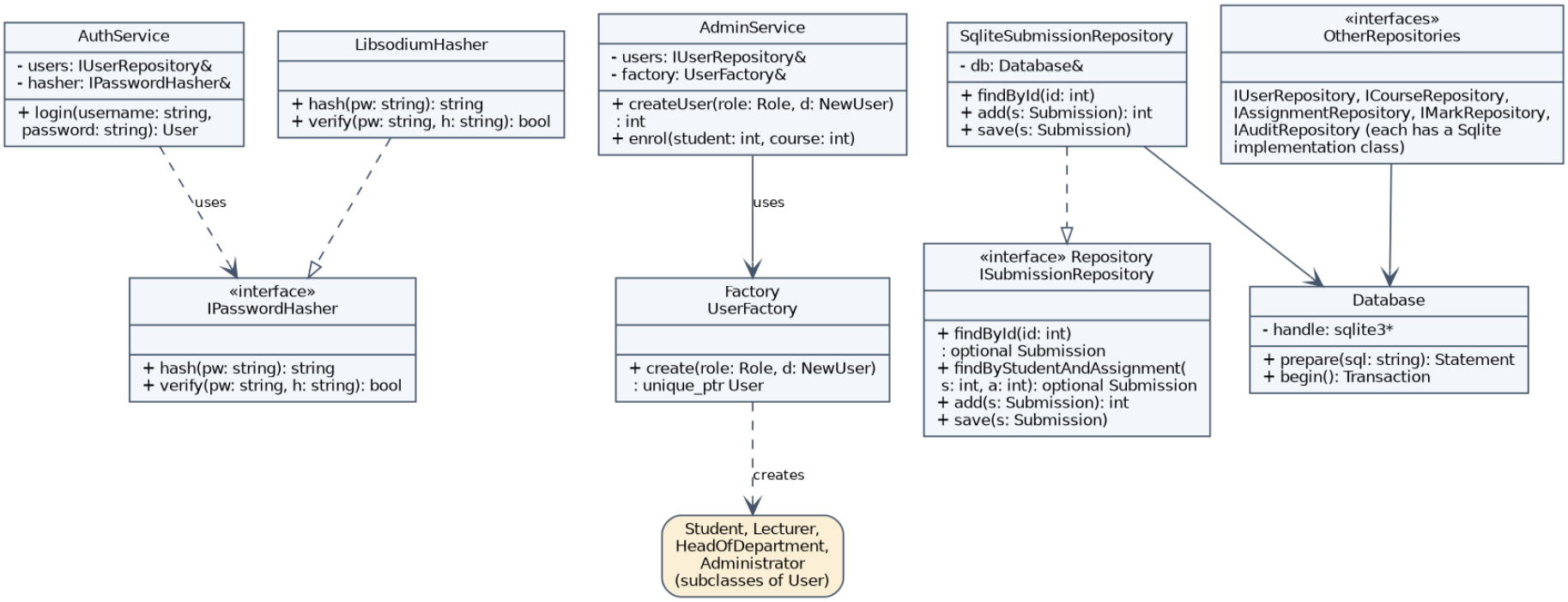


Figure 6: Factory, password hashing and Repository patterns

5.3 Behavioural models

The life cycle of a submission is a state machine with five states: Submitted or Late on arrival, then Marked, Moderated and Released (Figure 7). A Late submission has no way back to Submitted, which is how the rule that late work cannot be replaced is enforced. Self-transitions cover resubmission, re-marking and the correction of a released mark, each with a guard and a recorded reason.

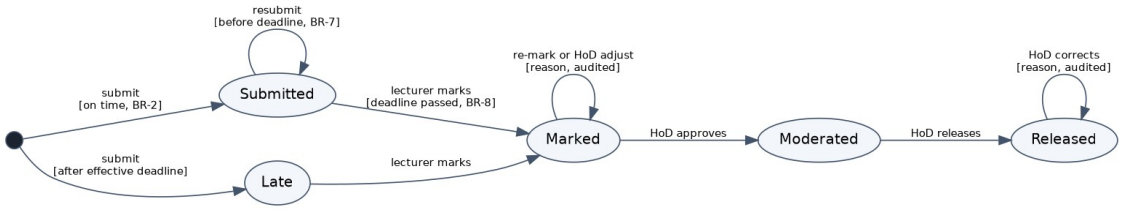


Figure 7: State machine diagram of the submission life cycle

The end-to-end process from submission to released mark is shown as an activity diagram with four swimlanes (Figure 8). The system lane makes visible that every state change and every mark change is done by the system, after validation.

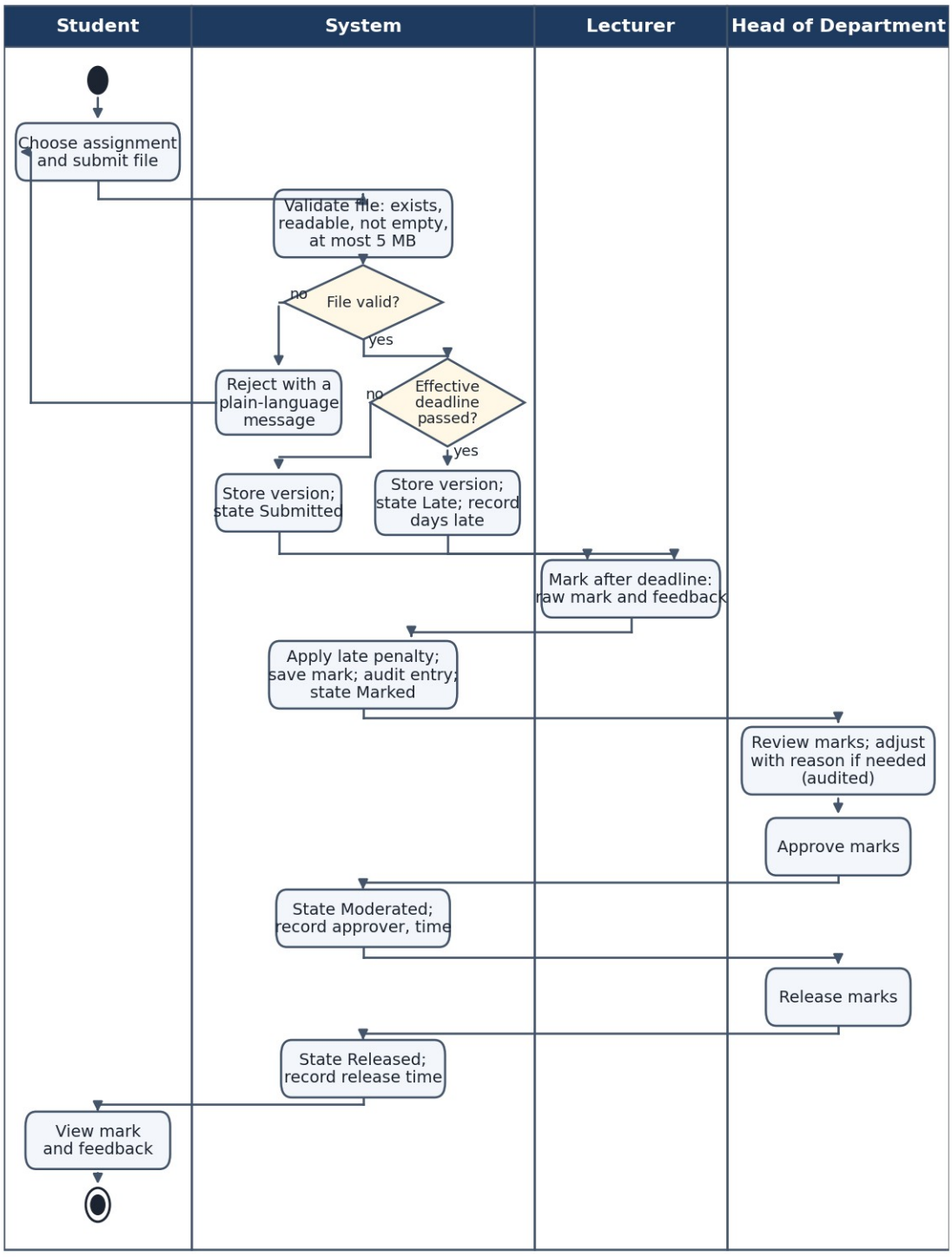


Figure 8: Activity diagram with swimlanes

Two sequence diagrams were drawn. Figure 9 shows *Submit assignment* with four paths in one fragment: an on-time first submission, a resubmission, a late first submission, and a refused attempt. Figure 10 shows *Moderate and release marks*, in which approval and release are each one transaction, so a batch is never applied halfway.

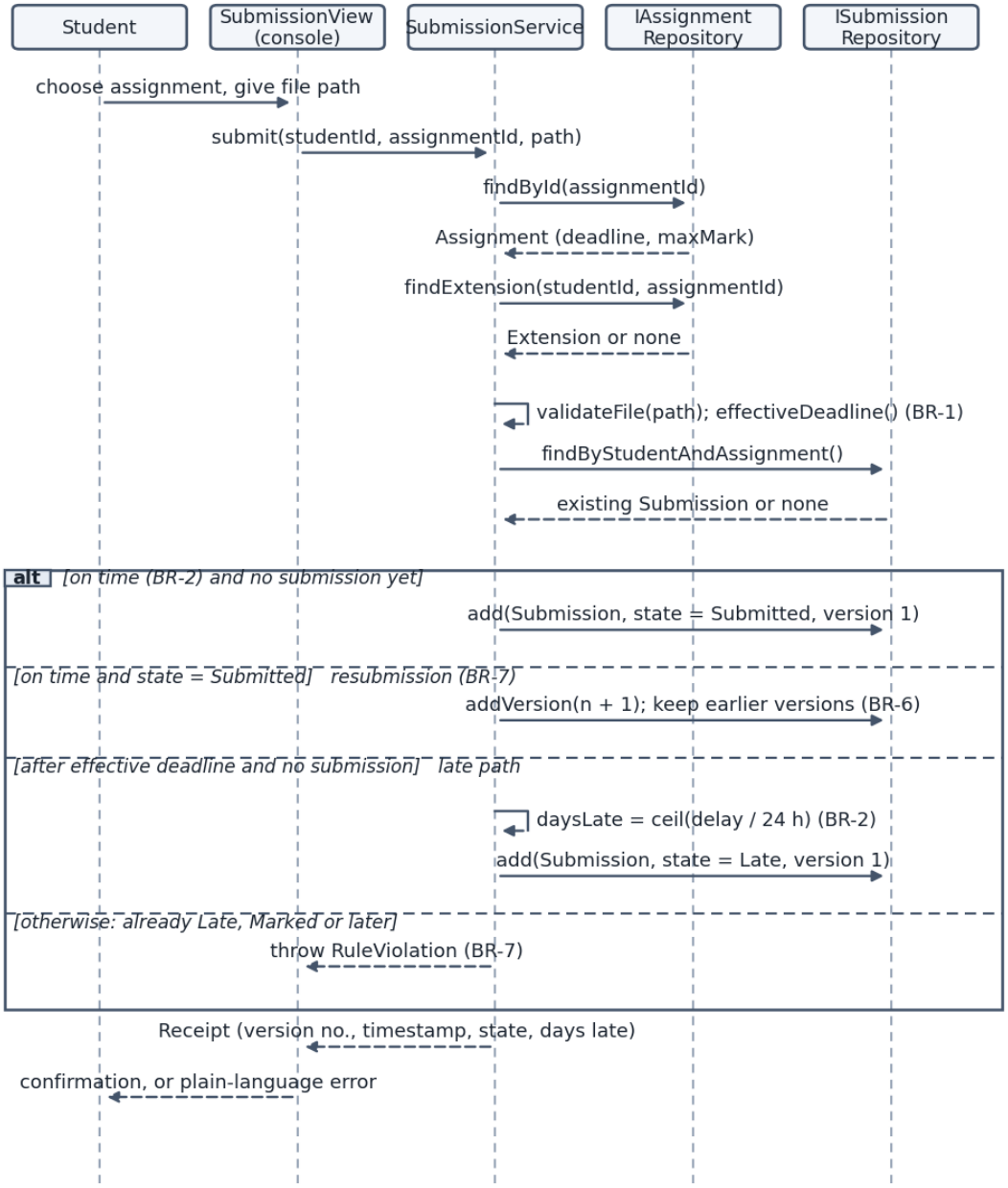


Figure 9: Sequence diagram for "Submit assignment"

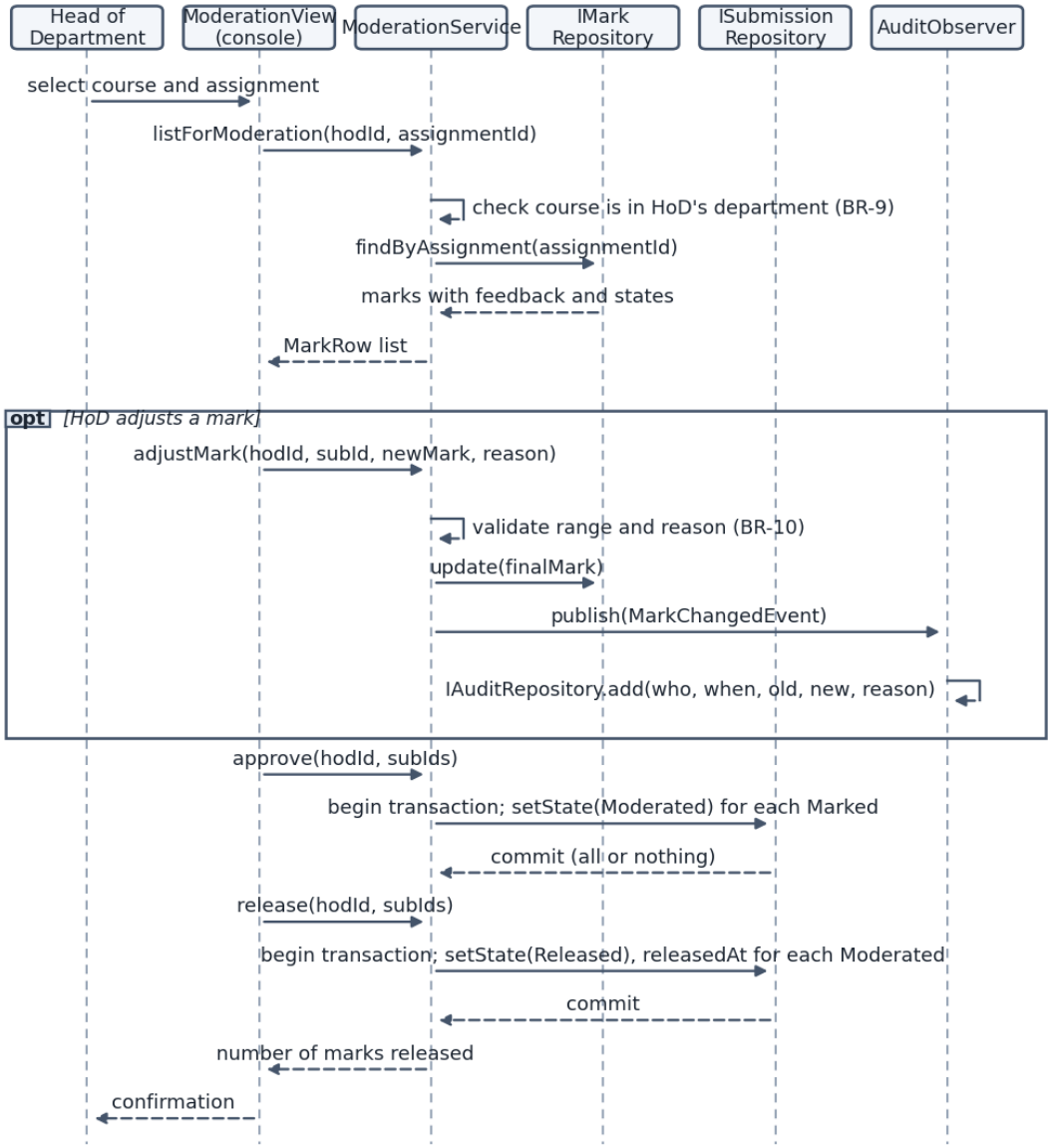


Figure 10: Sequence diagram for “Moderate and release marks”

5.4 Database design

The entity-relationship diagram (Figure 11) has eleven tables. The data dictionary and the full SQL schema are in Appendix E.

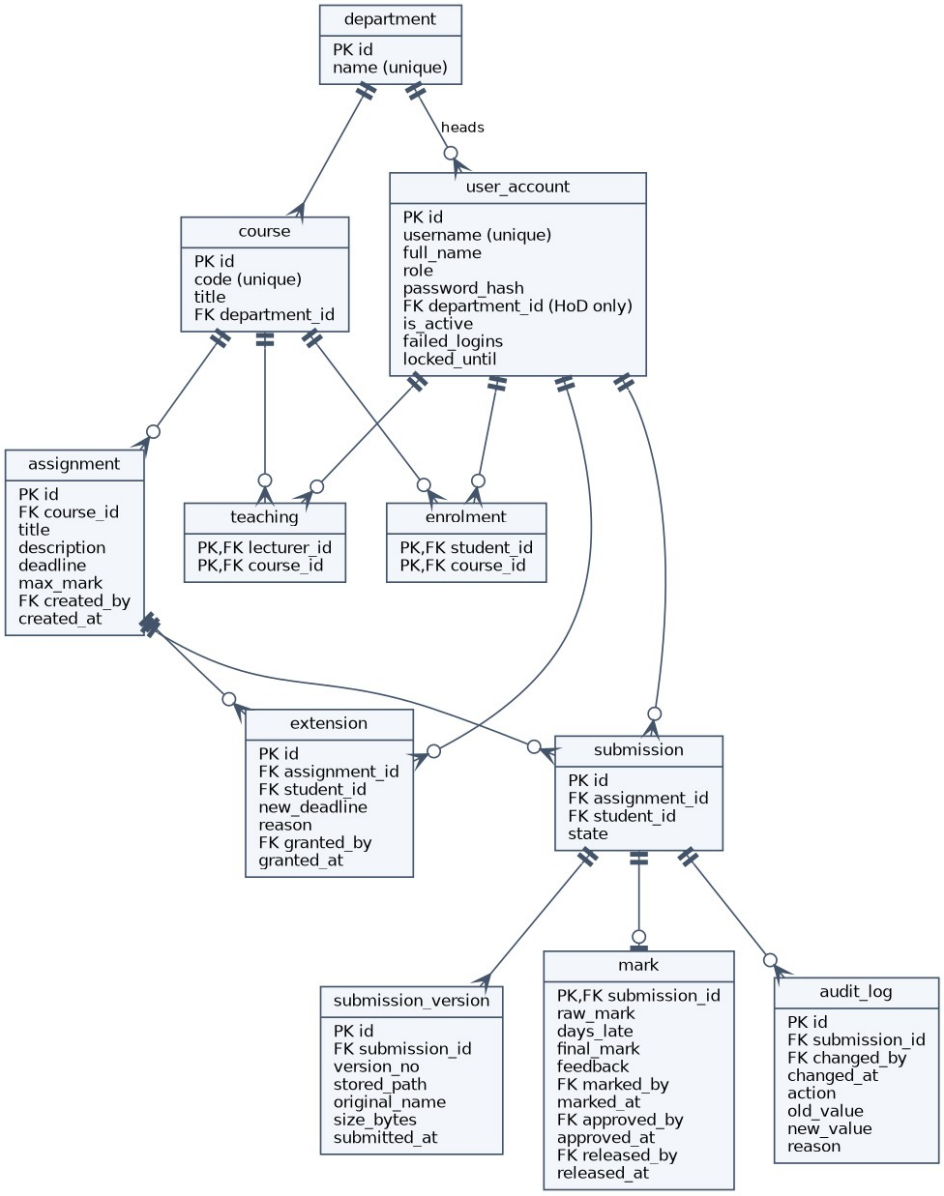


Figure 11: Entity-relationship diagram

The schema is in third normal form. Every column holds one value; submission versions are in their own table instead of repeated columns; many-to-many links (students to courses, lecturers to courses) are association tables with composite keys, so no column depends on part of a key; and the department name is stored once, so no non-key column depends on another. The current version of a submission and the penalty are derived values and are not stored. The `mark` table does store `days_late` and `final_mark`, because they record the result of the rule at marking time and must not change if the rule changes later. Their value depends on the assignment's maximum mark and the submission time, which are in other tables, so the table stays in third normal form.

The mapping from classes to tables differs from a one-to-one mapping in three ways. The four user classes share one table, `user_account`, with a role column and a CHECK that only a head of department has a department, because the roles share almost every attribute and login needs one lookup. The enumerations are stored as text protected by CHECK constraints. And constraints that span tables, such as

“the student is enrolled in the course of the assignment”, are enforced in the services and covered by tests. Append-only triggers on `audit_log` make it impossible for any function to edit or delete an audit entry.

6 Implementation

6.1 Structure and key decisions

The code is organised exactly as the design: 14 files under `src/` with 2,304 non-comment lines. The files map to layers: `domain` (users, entities, penalty Strategy, Observer, Factory), `data` and `schema` (SQLite wrapper, unit of work, repositories), `security` (hasher), `services` (seven services), and `ui` and `main` (console). It follows a coding standard that fixes naming, layering, SQL, password and error rules, and builds with CMake using `-Wall -Wextra -Wpedantic` and no warnings.

Four decisions shaped the implementation. First, every service method receives its collaborators as interfaces, so the same code runs against SQLite in production and in tests. Second, a unit-of-work interface (`IUnitOfWork`) wraps multi-row changes in a transaction that rolls back unless it is committed. It nests, so a service can call another and the whole change is still atomic. This is how batch release and the audit entry stay all-or-nothing. Third, time is stored as UTC in ISO 8601 text and as seconds in memory, and converted to local time only at the console, which makes deadline arithmetic independent of the time zone. Fourth, the late penalty is computed by integer arithmetic on seconds: days late is the number of started 24-hour periods, and marks are rounded half up to two decimals.

The console uses table-driven menus, one small action for each menu line, and a single loop that catches `LmsError` and shows its message, so invalid input never ends the program. All business errors are subclasses of `LmsError` (validation, access denied, rule violation, not found, database), and their messages are written for school users, for example “Marking opens after the deadline”.

Testability was designed in. Three seams matter: the clock, which lets a test place a submission exactly at, one second after, or 72 hours after a deadline; the file store, which a test can replace by one that fails; and the observer list, where a test can add an observer that throws, to prove that a mark cannot be saved without its audit entry. The same seams keep the production code free of test-only branches.

6.2 Security

Passwords are never stored. The system stores a salted Argon2id hash produced by libsodium (Denis, n.d.). The choice was made for three reasons. Argon2id won the Password Hashing Competition and is specified in RFC 9106 (Biryukov et al., 2021); it is memory-hard, which makes attacks with graphics cards or special hardware expensive; and the OWASP guidance names it as the first choice for new systems (OWASP, n.d.). Libsodium is a widely reviewed library, so no hashing code was written by hand, and it generates a random salt for each password and stores the parameters inside the hash, so they can be raised later. The interactive parameters (64 MiB of memory, two passes) give a login time of about 80 ms, far below the 2-second target. A test confirms that the same password gives different hashes for two users and that no stored hash contains the clear password.

Role-based access control is enforced in the services, not only in the menus: the first line of each service method calls `requireAllowed`, and a head of department and a lecturer can reach only their own department and courses. An account is locked for five minutes after five wrong passwords, and a wrong password and an unknown username give the same message, so the login screen does not reveal which usernames exist. All 40 SQL statements are prepared and bound, so user text is never part of the SQL (OWASP, n.d.), and a test stores the text `x'; DROP TABLE user_account; --` as a username without harm.

6.3 Persistence

Data is stored in a single SQLite file (SQLite Consortium, n.d.) accessed only through the repositories. Foreign keys are switched on, the busy timeout is five seconds, and writes use `BEGIN IMMEDIATE`, so that up to five simultaneous sessions wait for each other instead of corrupting data. Submitted files are copied into an application folder under a generated name, and the database stores the path, size and original name. Backup uses the SQLite online backup interface and is available to the administrator. Every mark change, whether by marking, re-marking, adjusting or correcting, is recorded with who, when, old value, new value and reason, and the log cannot be edited.

6.4 Fidelity to the design

Four small differences from the design were made on purpose and recorded. `AssignmentService` also uses the mark and user repositories (a defect found while building the extension screen, DEF-01), and since CR1 the unit of work; `AdminService` holds the database object for the backup; an `IUnitOfWork` replaced the simple RAII transaction; and the file store became an interface. The database in the code is identical to the schema in the design document.

7 Verification and Validation

7.1 Test strategy

The test plan follows the V-Model. Unit tests verify the detailed design, integration tests verify the architecture, system tests verify the SRS and the use case specifications, and acceptance scenarios validate the stakeholder statements. Each level has entry and exit criteria (Table 3). Test cases were designed with equivalence partitioning and boundary value analysis, most importantly for the deadline (one second before, at, and one second after the deadline; exactly 24 hours late and 24 hours plus one second; exactly 72 hours and 72 hours plus one second) and for marks, file sizes, usernames and maximum marks.

Table 3: V-Model levels with entry and exit criteria

Level	Verifies	Entry criterion	Exit criterion
Unit	Detailed design	Code compiles with no warnings	All pass; coverage of business logic at least 70%
Integration	Architecture	Unit tests pass	All pass; no open high-severity defect
System	SRS and use case specifications	Integration tests pass	Every flow of the three detailed use cases has a test; NFR targets met
Acceptance	Stakeholder statements	System tests pass	Each concern demonstrated or listed as unverified

Equivalence classes were chosen for each input. For a login they are a valid user, an unknown user, a wrong password, an inactive account and a locked account. For a submission they are an enrolled student or not, a file that is valid, missing, empty or too large, and each of the five states of an existing submission. For a mark they are a value inside the range, on its two boundaries and outside it, and a feedback text that is present or blank. This is why the system tests contain many small cases instead of a few long scenarios: each case isolates one rule.

7.2 Automated tests and coverage

The tests use GoogleTest and run with one command. There are 9 unit tests, 46 integration tests, 39 system tests and 10 tests for change request CR1, which makes 104 tests in total, and all pass. The full results are in Appendix B. Line coverage measured with gcov is 98.6% (546 of 554 lines) for the business-

logic layer (domain, services and common), against a target of 70%, and 92.5% for all code except the entry point. The few uncovered lines are defensive branches. The test code itself uses a fixed clock, an in-memory database and a failing file store or failing observer to simulate faults, which is how the exception flows of the use cases are exercised.

7.3 System and acceptance testing

System tests were derived directly from the three detailed use case specifications and cover every flow, including the exception flows: for example, a submission by a student who is not enrolled, an empty or oversized file, a resubmission of late work, a mark outside the permitted range, a database error during marking, and a batch release that fails halfway. A database error leaves nothing half-saved, because the tests show that the mark and its audit entry are both rolled back. Robustness tests apply 60 invalid inputs to every service and confirm that each is rejected with a plain message and that none crashes the program (NFR-06).

Table 4 summarises the acceptance scenarios derived from the stakeholder statements.

Table 4: Acceptance scenarios

Stakeholder concern	Result
Principal: no mark visible before the HoD has checked it	Demonstrated: a student sees nothing while a mark is Marked or Moderated
Lecturer: extensions for students	Demonstrated, individually and for a whole class (CR1)
HoD: 10% per day for three days, then zero; sees every mark of the department	Demonstrated at all boundaries
Student representative: resubmit before the deadline; written feedback; late work not rejected	Demonstrated
IT officer: offline, protected passwords, fast with 800 students	Demonstrated: no network code, Argon2id, login 78 ms
Principal: simple for non-IT teachers	**Not verified**: needs a user test with three first-time lecturers

7.4 Performance and defects

With 800 student and 60 staff accounts in a real database file, a login took 78 ms, listing 150 submissions 0.5 ms, saving a mark 2.4 ms, and a class extension for 800 students 32 ms, all far below the targets of 1 to 2 seconds. A test with five simultaneous database connections made 100 concurrent writes with no failure, and the integrity check passed.

Seven defects were logged (Table 5). None was of high severity, and no defect was found in the late-penalty rule or the moderation flow. Two were genuine product problems found by the work itself: a design gap (DEF-01, found when building the extension screen) and a regression introduced during CR1 (DEF-05, caught by the existing console test).

Table 5: Defect log (severity, cause, fix and retest result)

ID	Found in	Severity	Cause	Fix and retest
DEF-01	Implementation	Medium	Design gap: no operation to list a course's students	Added `studentsIn()`; tests pass
DEF-02	Unit test	Low	Wrong expected value in a test	Corrected; passes
DEF-03	Test build	Low	Missing include	Added; builds
DEF-04	Metrics	Medium	Seven functions above complexity 10 (menus up to 23)	Table-driven menus, extracted validators; all tests pass

ID	Found in	Severity	Cause	Fix and retest
DEF-05	CR1 regression	Medium	New menu item renumbered existing options	Moved to position 7; 93 of 93 pass
DEF-06	Static analysis	Low	Five groups of style findings	Fixed; cppcheck reports nothing
DEF-07	Review	Low	No automated test for NFR-10	Added concurrency test; passes

7.5 Limits of the evidence

The following were not done and should not be assumed: a usability test with real lecturers (NFR-08); testing on Windows or on the school's old computers; a run of the pipeline on a hosted service (the same commands were run locally); and a rehearsal of a restore on site. They remain open in the risk register.

8 Configuration Management and Quality Assurance

8.1 Configuration management

The source code is under Git with a documented branching strategy based on GitHub flow. The `main` branch always builds and passes the tests; work is done on short-lived feature branches and merged by pull request with a review checklist; releases are annotated tags in the form `vMAJOR.MINOR.PATCH`. The repository has 24 commits, nine merged pull requests, and two tagged releases (v1.0.0 and v1.1.0). Commit messages state the requirement or reason for the change. The change request followed the same path: a branch, impact analysis, tests, and a merge before the tag.

The continuous-integration pipeline is defined as a GitHub Actions workflow with two jobs run on every push. The first builds the project and runs all tests. The second runs the static analyser, which must report nothing, builds with coverage instrumentation, and fails if the line coverage of the business-logic layer is below 70%. Because the pipeline was run here by executing the same commands locally, its hosted runs are not claimed as evidence.

Version numbers show the kind of change. The first release is 1.0.0; the change request that added behaviour raised the minor number to 1.1.0; a corrective fix would raise only the patch number. Each tag marks a state of the repository that was built and tested from a clean clone, and the release notes list new features, changes, upgrade steps and known limitations. A change request starts with an impact analysis and ends with updated documents, new tests and a full regression run, so that a change is never merged on the strength of a quick manual check.

8.2 Quality assurance

The cppcheck analyser (warnings, style, performance and portability checks) found style problems in five groups, which were all fixed, and now reports nothing for the source and tests. Code metrics were measured with lizard: 2,304 lines of code, 213 functions, an average of 7 lines and a cyclomatic complexity of 2.5 per function. The first measurement showed seven functions above the coding standard's limit of 10, the worst being the console menus (23); they were refactored into table-driven menus and extracted validators, leaving two functions at 13 and 11, which are flat sequences of tested conditions and were accepted.

A self code review used a twelve-point checklist covering SQL construction, the data-layer boundary, password handling, role checks, atomicity, resource release, error messages, function size, naming, constructors, fidelity to the design, and duplication. It led to the extracted validators, the table-driven menus, explicit constructors, consistent parameter names and the merging of duplicated menu code. The full record is in the repository's quality report.

9 Deployment

The system is deployed on one school computer running Ubuntu 22.04 or later, or Windows 10 with WSL2, as in the UML deployment diagram (Figure 12). The program, the SQLite database file and the folder of submitted files are on that computer, with the two libraries it uses. An optional variant shares the database folder on the local network for light use by up to five sessions, and backups go to a USB drive or a second computer.

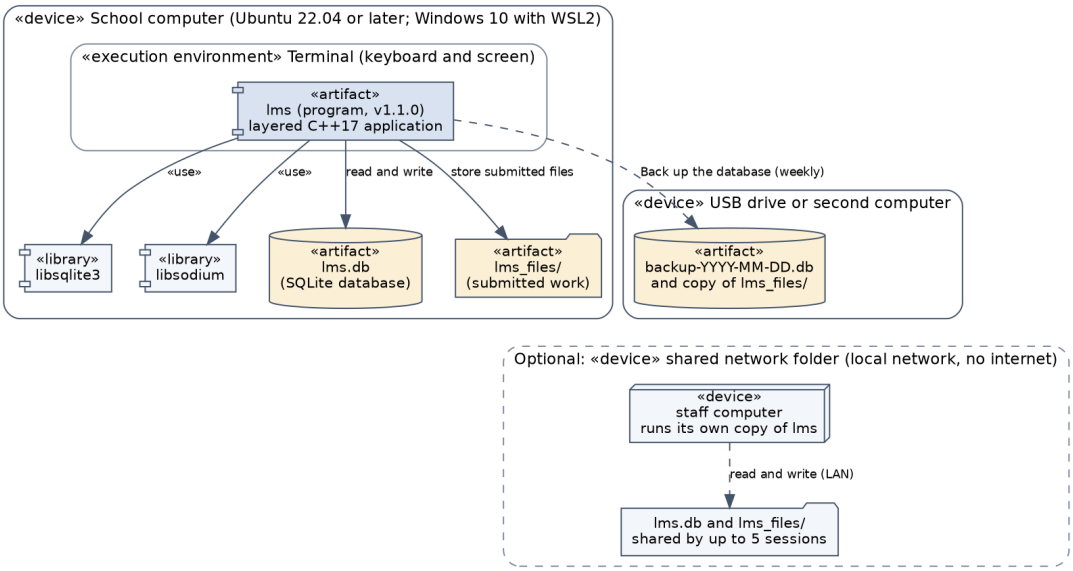


Figure 12: UML deployment diagram

The installation guide is written for a school technician and takes about thirty minutes: install the tools, build, copy the program to a permanent folder, start it, create the first administrator, and check the installation by creating a department. It includes a weekly backup routine, a ten-minute restore procedure, an update procedure and a table of the common messages with what to do. A user manual for each role (Appendix C) and release notes accompany the release, which was tagged v1.0.0. Risks that remain are that the build has been tried only on one platform and that the restore procedure has not yet been rehearsed on the school’s computers.

A go-live checklist helps the technician. Install and test the program on the school computer; create the first administrator and the departments; create the accounts and enrol students by course; run a rehearsal assignment through submission, marking, moderation and release with a test student; take the first backup and rehearse a restore; and give each role its user manual. Training for lecturers should be short because the menus use plain words, and it should end with the lecturer creating a real assignment, which is also the measurement used by the usability requirement.

10 Maintenance: Change Request CR1

After release the school submitted change request CR1: a lecturer should be able to extend the deadline for a whole class, and extensions should be shown on the student’s assignment list.

10.1 Classification and impact analysis

The change is **perfective maintenance** (ISO/IEC/IEEE 14764, 2022): the system works as specified and the users ask for improved functionality. It is not corrective (there is no fault), adaptive (the environment is unchanged) or preventive (nothing is restructured to avoid future faults). Table 6 lists everything affected.

Table 6: Impact analysis for CR1

Kind	Affected items	Effect
Requirements	FR-10 (modified), FR-06 (modified), FR-24 (new); BR-1 and BR-10 unchanged and still apply	One new requirement; two modified
Use cases	UC08 (new alternative flow), UC17 (shows extension), UC22 (new, extends UC08)	Diagram and specifications updated
Classes	`AssignmentService` (new operation; new dependency `IUnitOfWork`), `AssignmentRow` (new field), `ClassExtensionResult` (new), `ConsoleApp` (menu item 7; list text)	Four classes touched
Tables	`extension`: rows only	No schema change and no data migration
Diagrams	Use case, class and component diagrams updated; new sequence diagram; state machine, activity and ERD unchanged	
Tests	All 93 existing tests re-run; 10 new tests for CR1	

10.2 Design and implementation

A class extension is stored as one `extension` row per enrolled student. A separate class-wide table would also cover students who enrol later, but needs a schema change and a migration, and the effective-deadline rule would have to look in two places. One row per student needs no schema change, keeps BR-1 and the penalty code as they are, and makes individual and class extensions combine naturally. Its cost, that students who join later are not covered, is documented and listed as a risk. The new operation `grantClassExtension` writes all rows in one transaction, never shortens a later extension a student already has, and returns how many students were changed and how many kept their later date (Figures 13 and 14).

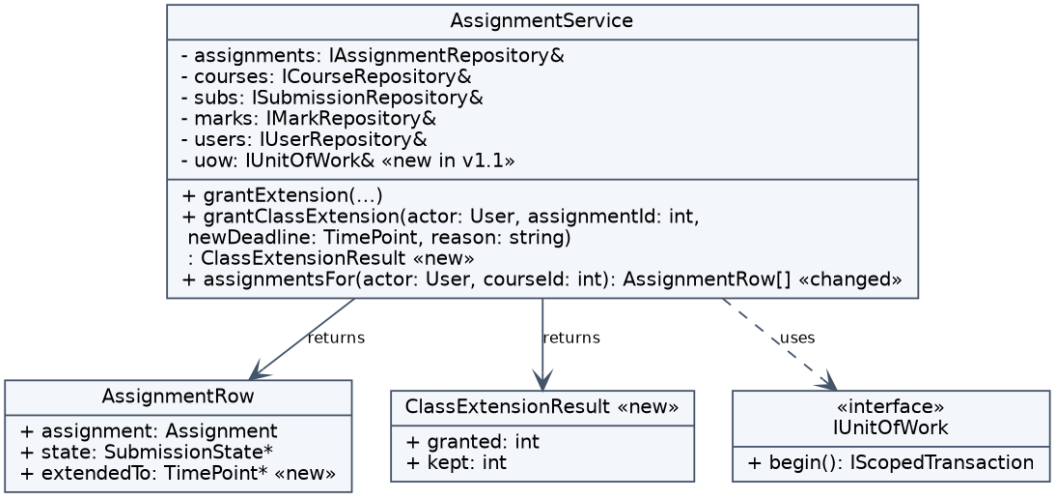


Figure 13: Updated design: AssignmentService, AssignmentRow and ClassExtensionResult

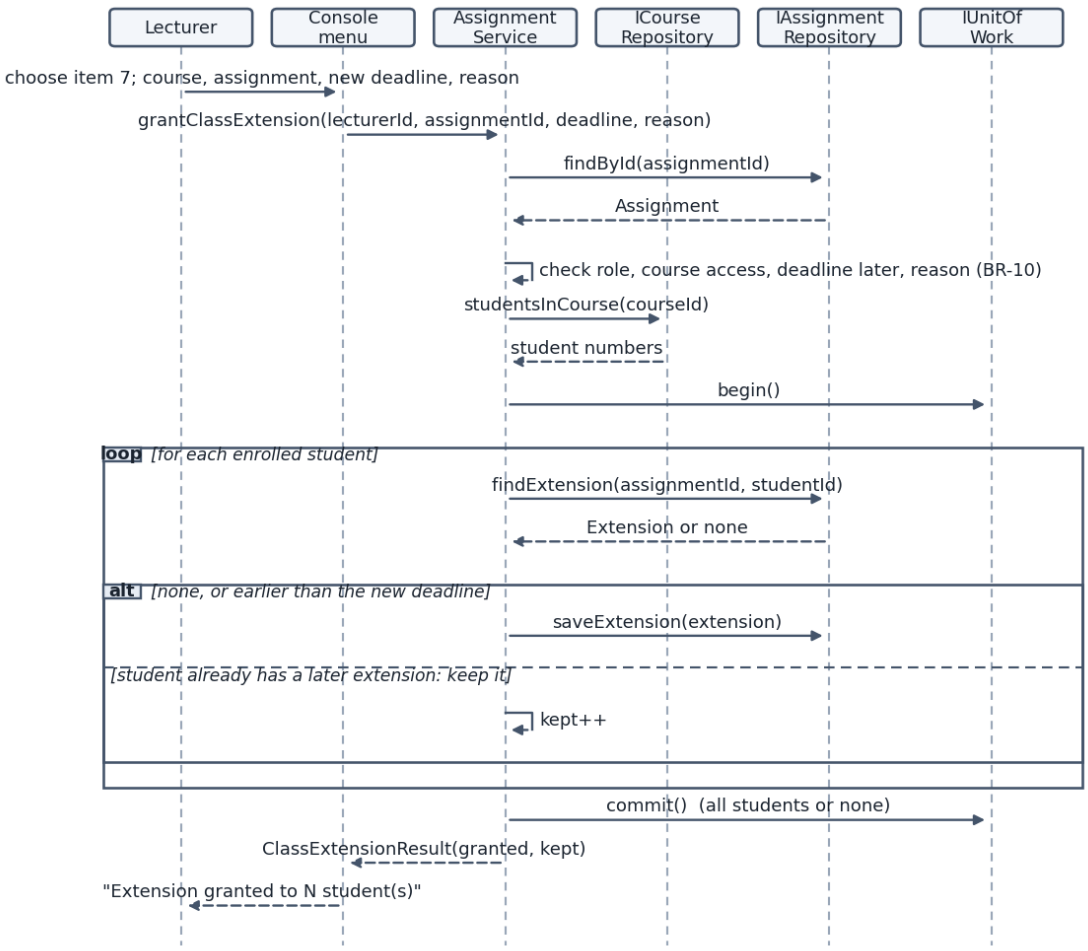


Figure 14: Sequence diagram for "Grant class extension"

The change was made on its own branch and released as v1.1.0. The student's list now shows `(extended to ...)` beside a deadline that has been extended, and the lecturer menu has a new item.

The size of the change is evidence of maintainability. It touched four classes, added one new type and one menu item, needed no database change, and left the penalty code, the repositories and the other services unchanged. This follows from an earlier design decision: the effective deadline was already computed from an extension row, so any way of creating those rows works with the penalty rule and the submission rules automatically.

10.3 Regression testing

All 93 tests that existed at v1.0.0 were run again on v1.1.0 without changing any test case, and all passed. Only the shared test fixture gained the new constructor argument. The regression run was not a formality: the first version of the change put the new menu item at position 4 and renumbered existing options, and a console test failed immediately (DEF-05). The item was moved to position 7 so existing numbers stay stable. Ten new tests cover class-wide extension for every enrolled student, the rule that a later extension is never shortened, the effect on penalties, validation and access, atomicity (a failure for one student rolls back all), the student's display, and the console flow. Performance was re-checked: an extension for 800 students takes 32 ms. The v1.1.0 release notes, user manual, traceability matrix and risk register were updated.

11 Professional, Legal and Ethical Issues

11.1 Data protection

The LMS holds the names, accounts, submissions, marks and feedback of learners, most of whom are children. For a school in Lesotho the relevant legislation is the Data Protection Act of 2011, and other jurisdictions have comparable laws; the school should confirm the current obligations with legal advice (Kingdom of Lesotho, 2011). The common principles are lawful and specific purpose, collection of only what is needed, accuracy, protection with appropriate security safeguards, limited retention, and the right of people to see and correct their data. The design addresses most of these. Only the data needed for assessment is stored (no addresses, identity numbers or contact details). Access is by role: a student sees only their own work and released marks, a lecturer only their courses, and a head of department only their department. Passwords are hashed. The audit log supports accuracy and accountability, because corrections are recorded with a reason. Two obligations are not yet met in version 1.1: there is no tool to delete data after a retention period, and no export of a learner's data on request (the optional CSV export, FR-23, was not built). Both are listed as future work, and until then the school must handle such requests through the administrator and the database file.

11.2 Security of assessment records

Assessment records are valuable and attractive to alter. Confidentiality is protected by role-based access, by the rule that a mark cannot be seen before release, and by hashed credentials. Integrity is protected by the audit log, which the database itself prevents from being edited, by transactions that make multi-step changes all-or-nothing, and by the rule that a change needs a reason. Availability is supported by the backup function and the restore guide. The main limit is that the database and the submitted files are not encrypted at rest, so anyone with access to the computer's files can read them. The installation guide therefore asks for operating-system permissions on the program folder and for a physical lock on the computer; encryption at rest is future work. A related professional point is that a single administrator account holds great power, so its use should be limited and its password kept in a safe place.

11.3 Accessibility

A text console works with keyboard only, uses no colour to convey meaning (statuses and errors are words such as LATE or RELEASED), and can be read by screen readers that read the terminal, and the user can enlarge the font of the terminal. Dates and menu choices are typed, which suits users who cannot use a mouse. The limits are that the system has not been tested with a screen reader or with users with disabilities, and that terminal screen readers vary by platform. A learner who cannot type a file path would need help, so a graphical file chooser is a sensible later improvement, and the school should consider adjustments for individual learners.

11.4 Fairness of the late-penalty rules

The rule of 10% of the maximum mark for each started 24 hours, up to three days and then zero, is clear, predictable and applied by the program in the same way to everyone, which removes the inconsistency of a person deciding case by case. It is also harsh in two ways. Work one second late loses a full day's penalty, and the drop from 30% to zero at 72 hours is a cliff; weekends count as days. These effects were not hidden: the rule is in the stakeholder resolutions, the student is told how many days late the work is when it is submitted, and the penalty is shown to the lecturer and the HoD before release. Fairness is also supported by features that let people respond to circumstances: individual and class-wide extensions with a recorded reason (so that a student with a good reason, or a class affected by a power cut, is not penalised), the HoD's right to adjust or correct a mark with a reason, and the Strategy pattern, which lets the school change the rule without changing the program. Late work is never rejected, and still receives

feedback even when the mark is zero. Automation also carries a risk: a penalty applied without thought could be unjust, so the design keeps human judgement in the loop through extensions and moderation.

Transparency and human oversight are the ethical basis of the design. People affected by a decision should be able to see how it was made, so the penalty rule is stated in the manuals, students are told on submission how late their work is, and marks carry written feedback. People should also be able to challenge a decision, so the HoD can adjust a mark before release and correct it afterwards, and every such change leaves a permanent record. Nothing in the system decides a mark automatically except the arithmetic of the published penalty rule.

11.5 Professional responsibility

The work follows professional codes in several ways (ACM, 2018; BCS, 2022). The system's limits are stated honestly, including the unverified usability requirement and the untested platforms. All dependencies have licences that allow use in a school. Decisions that affect other people, such as the resolutions of the stakeholder conflicts, are recorded with reasons so that the school can review and change them. The software was designed for the school's real needs and constraints, such as old computers and no internet, not for the developer's convenience.

12 Conclusion

The project delivered a working Moderated LMS and took it through every phase of the life cycle: a feasibility-based plan, a requirements specification that resolves the conflicts between five stakeholders, a layered design with four patterns and a normalised database, an implementation that closely follows the design, a verification effort of 104 automated tests with 98.6% coverage of the business logic, and a maintenance release that was checked by a full regression run. The central rule, that students never see a mark before the head of department has released it, is enforced in the domain and tested at several levels, and every mark change is recorded in a log that cannot be edited.

Three lessons stand out. Designing for testability (interfaces, a replaceable clock, a unit of work) paid for itself many times, because every exception flow of the use cases could be tested and the regression suite caught a real mistake in the change request. Writing the stakeholder conflicts down as explicit, numbered decisions made later work faster and the assumptions easy to review. And a traceability matrix, built from test names and requirement identifiers, showed immediately which requirement had no test: it exposed the missing concurrency test and the one requirement that was dropped.

The main limits are the missing usability test with real lecturers, the untested platforms and school hardware, the lack of data-retention and data-export tools, and the lack of encryption at rest. Future work should address these first, followed by a graphical interface, the cover of students who join after a class extension, and a configurable penalty rule.

13 References

ACM (2018) *ACM Code of Ethics and Professional Conduct*. New York: Association for Computing Machinery.

BCS (2022) *Code of Conduct for BCS Members*. Swindon: BCS, The Chartered Institute for IT.

Biryukov, A., Dinu, D., Khovratovich, D. and Josefsson, S. (2021) *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106. Internet Engineering Task Force.

Denis, F. (n.d.) *Libsodium documentation: password hashing*. Available at: https://doc.libsodium.org (Accessed: during development).

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering*. Geneva: ISO.

ISO/IEC/IEEE (2022) *ISO/IEC/IEEE 14764:2022 Software engineering: Software life cycle processes: Maintenance*. Geneva: ISO.

Kingdom of Lesotho (2011) *Data Protection Act, 2011*. Maseru: Government of Lesotho.

Martin, R.C. (2003) *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ: Prentice Hall.

OWASP (n.d.) *Password Storage Cheat Sheet and SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org (Accessed: during development).

Royce, W.W. (1970) ‘Managing the development of large software systems’, *Proceedings of IEEE WESCON*, pp. 1-9.

SQLite Consortium (n.d.) *SQLite documentation*. Available at: https://www.sqlite.org/docs.html (Accessed: during development).

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

14 Appendices

The appendices are the traceability matrix (A), the full test results (B), the user manuals (C), the updated risk register (D), and the data dictionary with the SQL schema (E). The source code, installation guide, release notes, test and quality reports and the defect log are in the Git repository.

Appendix A: Requirements traceability matrix

Each functional requirement is linked to its use case, design elements, source files (in `src/`) and automated tests (suite.test). Requirements FR-01 to FR-23 are version 1.0; FR-24 was added by CR1.

Table A1: Functional requirements traceability

Req.	Pri.	Use case	Design elements	Source	Tests	Result
FR-01	Must	UC01	AuthService, User::allows(), ConsoleApp	services.cpp, domain.cpp, ui.cpp	Auth.LoginSucceedsAndIsCaseInsensitiveOnUsername; Auth.WrongPasswordAndUnknownUserGiveTheSameMessage; Auth.DeactivatedAccountCannotLogIn; Domain.PermissionMatrixPerRole; Console.FailedLoginShowsAPlainMessageAndQuitWorks	All pass
FR-02	Must	UC03	AdminService::createUser, UserFactory, user_account	services.cpp, domain.cpp, data.cpp	Admin.CreateUserValidatesEveryField; Admin.HeadOfDepartmentRules; Domain.UserFactoryBuildsEachRoleAndValidates; Console.AdministratorCreatesAnAccount	All pass
FR-03	Must	UC04	AdminService, department, course	services.cpp, data.cpp	Admin.DepartmentsCoursesEnrolmentsAndAssignments; Data.RepositoriesRoundTripEntities	All pass
FR-04	Must	UC05	AdminService::enrol, assignLecturer; enrolment, teaching	services.cpp, data.cpp	Admin.DepartmentsCoursesEnrolmentsAndAssignments	All pass
FR-05	Must	UC07	AssignmentService::createAssignment; assignment	services.cpp, data.cpp	AssignmentSvc.CreateAssignmentValidation	All pass
FR-06	Must	UC17	AssignmentService::coursesFor, assignmentsFor	services.cpp, ui.cpp	AssignmentSvc.ListingsRespectRoles; CR1.StudentsSeeTheirExtensionOnTheAssignmentList; CR1.ConsoleShowsNoExtensionTextWhenThereIsNone; Console.StudentSubmitsWork	All pass
FR-07	Must	UC18	SubmissionService::submit, LocalFileStore; submission, submission_version	services.cpp, data.cpp	Submission.OnTimeFirstSubmission; Submission.InvalidFilesAreRejectedAndStoreNothing; Submission.OnlyEnrolledStudentsMaySubmit; UC18.Main_OnTimeSubmission; UC18.E1_StudentNotEnrolled; UC18.E2_InvalidFile; UC18.E5_StorageErrorLeavesNoPartialVersion; Submission.DatabaseFailureLeavesNoPartialSubmission	All pass
FR-08	Must	UC19	SubmissionService::submit (resubmission path); submission_version	services.cpp	Submission.ResubmissionBeforeDeadlineKeepsAllVersions; Submission.CannotResubmitAfterDeadlineOrAfterLateOrAfterMarking; UC18.A1_ResubmissionBeforeDeadline; UC18.E4_SubmissionAfterMarkingRefused	All pass
FR-09	Must	UC20	SubmissionService::submit (late path); state Late	services.cpp	Submission.DeadlineBoundaries; UC18.A2_LateSubmissionIsAcceptedAndFlagged; UC18.A2_SubmissionAfterSeventyTwoHoursIsStillAccepted; UC18.E3_ResubmissionOfLateWorkRefused; Console.LateSubmissionWarnsAboutPenalty	All pass
FR-10	Must	UC08	AssignmentService::grantExtension; extension	services.cpp, data.cpp	AssignmentSvc.ExtensionRules; Data.ExtensionIsUpsertedPerStudentAndAssignment; Submission.ExtensionMovesTheEffectiveDeadline; Marking.ExtensionRemovesThePenalty	All pass

Req.	Pri.	Use case	Design elements	Source	Tests	Result
FR-11	Must	UC11	ILatePenaltyPolicy, TenPercentPerDayPolicy	domain.cpp	Penalty.BoundaryValuesForDeadlineAndDays; Penalty.MarkBoundariesAndFloorAtZero; Penalty.RoundsHalfUpToTwoDecimals; Penalty.PolicyIsReplaceable; Marking.LatePenaltyIsApplied; UC10.Main_LatePenaltyShownAndApplied	All pass
FR-12	Must	UC09, UC10	MarkingService::mark, remark; AssignmentService::listSubmissions; mark	services.cpp, data.cpp	Marking.CannotMarkBeforeTheEffectiveDeadline; Marking.OnTimeMarkHasNoPenaltyAndWritesAnAuditEntry; Marking.RejectsInvalidMarksFeedbackAndWrongPeople; UC10.Main_MarkAfterDeadlineHiddenFromStudent; UC10.E1_MarkOutsideRange; UC10.E2_FeedbackMissing; UC10.E3_DeadlineNotYetPassed; UC10.E4_ModeratedOrReleasedMarkCannotBeChangedByLecturer; Console.LecturerMarksAndHodModeratesAndReleases	All pass
FR-13	Must	UC12	MarkChangePublisher, AuditObserver; audit_log (append-only triggers)	domain.cpp, services.cpp, data.cpp	Data.AuditLogIsAppendOnly; Marking.RemarkRequiresReasonAndIsAudited; Marking.FailedAuditRollsBackTheMark; Moderation.AdjustMarkBeforeApprovalIsAuditedAndValidated; UC10.A1_RemarkWithReason; UC10.E5_DatabaseErrorLeavesNothingSaved	All pass
FR-14	Must	UC13	ModerationService::listForModeration	services.cpp	Moderation.ReviewIsLimitedToOwnDepartment; UC14.E2_CourseOutsideDepartment	All pass
FR-15	Must	UC14, UC15	ModerationService::approve, adjustMark; state Moderated	services.cpp	Moderation.AdjustMarkBeforeApprovalIsAuditedAndValidated; Moderation.ApproveAndReleaseBatchesSkipWrongStates; UC14.A1_AdjustAMark; UC14.E1_UnmarkedSubmissionIsSkipped; UC14.E3_ReasonMissingOrMarkOutOfRange	All pass
FR-16	Must	UC14	ModerationService::release; state Released	services.cpp	Moderation.ReleaseRequiresApprovalFirst; Moderation.BatchIsAllOrNothing; UC14.Main_ApproveThenRelease; UC14.A2_PartialRelease; UC14.E4_BatchRolledBackOnDatabaseError	All pass
FR-17	Must	UC21	SubmissionService::releasedMark	services.cpp	Moderation.StudentsNeverSeeUnreleasedMarks; Console.StudentSeesNothingBeforeRelease; UC10.Main_MarkAfterDeadlineHiddenFromStudent	All pass
FR-18	Must	all	requireAllowed(), User::allows()	services.cpp, domain.cpp	Domain.PermissionMatrixPerRole; AccessControl.EachRoleIsLimitedToItsOwnFunctions; AccessControl.DeactivatedUserCannotPerformActions	All pass
FR-19	Should	UC14	ModerationService::correctReleasedMark	services.cpp	Moderation.CorrectionOfAReleasedMarkIsAuditedAndVisible; UC14.A3_CorrectAReleasedMark	All pass
FR-20	Should	UC16	AuditService::list; audit_log	services.cpp, data.cpp	Moderation.AuditLogAccessIsRoleBased; Data.AuditFilterByDepartmentUserAndTime; Console.ChangePasswordAndAuditLogScreens	All pass

Req.	Pri.	Use case	Design elements	Source	Tests	Result
FR-21	Should	UC02	AuthService::changePassword, lockout	services.cpp	Auth.AccountLocksAfterFiveFailuresAndUnlocksAfterFiveMinutes; Auth.SuccessfulLoginResetsTheFailureCounter; Auth.ChangePasswordChecksOldPasswordAndRules; Console.ChangePasswordAndAuditLogScreens	All pass
FR-22	Should	UC06	AdminService::backup, Database::backupTo	services.cpp, data.cpp	Data.BackupCreatesAnOpenableCopy	All pass
FR-23	Could	-	(CSV export) not built	-	-	Descoped (Could have)
FR-24	Must (CR1)	UC22	AssignmentService::grantClassExtension; extension rows	services.cpp	CR1.ClassExtensionGivesEveryEnrolledStudentTheNewDeadline; CR1.NeverShortensALaterIndividualExtension; CR1.ExtendsAnEarlierIndividualExtension; CR1.ClassExtensionRemovesThePenaltyWhenMarking; CR1.ValidationAndAccessRules; CR1.ClassExtensionIsAllOrNothing; CR1.ConsoleLecturerGrantsAClassExtensionAndStudentsSeeIt; CR1.ConsoleReportsStudentsWhoKeepALaterExtension	All pass

Table A2: Non-functional requirements verification

Req.	Pri.	Method	Evidence	Result achieved	Status
NFR-01	Must	Test	Performance.LoginListingAndSavingMeetTheTargetsWithEightHundredStudents	Login 78 ms with 800 students and 60 staff stored (target 2,000 ms)	Met
NFR-02	Must	Test	Performance.LoginListingAndSavingMeetTheTargetsWithEightHundredStudents	Listing 150 submissions 0.5 ms; saving a mark 2.4 ms; class extension for 800 students 32 ms (target 1,000 ms)	Met
NFR-03	Must	Inspection	(no test: nothing to run)	No networking calls exist in src/ (search for socket, curl, http, connect, getaddrinfo found nothing); every test runs without a network	Met
NFR-04	Must	Test, inspection	Auth.PasswordsAreStoredAsSaltedHashesOnly	Argon2id hashes; same password gives different hashes; no stored hash contains the clear password	Met
NFR-05	Must	Inspection, test	Data.QueriesAreParameterisedAgainstInjection	All 40 statements in data.cpp are prepared and bound; no SQL outside data.cpp and schema.cpp; injection text is stored as data	Met
NFR-06	Must	Test	Robustness.AtLeastFiftyInvalidInputsAreRejectedCleanly	60 invalid inputs, a damaged database, bad console input and end of input all produce a plain error or a clean exit, no crash	Met
NFR-07	Must	Test	Data.AuditLogIsAppendOnly	Every mark change writes one audit entry (marking, re-marking, adjusting, correcting); update and delete on the log are refused by the database	Met

Req.	Pri.	Method	Evidence	Result achieved	Status
NFR-08	Must	User test	(not run)	Needs three first-time lecturers using the manual; not possible without real users. Menu text was reviewed for plain language	Not yet verified
NFR-09	Should	Inspection	Console tests	Status and errors are text; no colour; keyboard only	Met
NFR-10	Should	Test	Concurrency.FiveSessionsWriteToOneDatabaseFileWithoutCorruption	Five connections, 100 concurrent writes, no failure, integrity check ok (run 3 times)	Met
NFR-11	Should	Measurement	All tests	Business-logic coverage 98.6% (target 70%); cppcheck reports nothing	Met
NFR-12	Should	Demonstration	Data.BackupCreatesAnOpenableCopy	A backup was written and opened in under one second; restore is a file copy (procedure in docs/INSTALL.md)	Met (procedure to be rehearsed on site)

Appendix B: Full automated test results (v1.1.0)

Run of `lms_tests` on Ubuntu 24.04 with GCC 13.3, SQLite 3.45.1 and libsodium 1.0.18: **104 tests, 104 passed, 0 failed**.

Table B1: All tests

#	Test (suite.test)	Result	Time (ms)
1	Penalty.BoundaryValuesForDeadlineAndDays	PASS	0
2	Penalty.MarkBoundariesAndFloorAtZero	PASS	0
3	Penalty.RoundsHalfUpToTwoDecimals	PASS	0
4	Penalty.PolicyIsReplaceable	PASS	0
5	Domain.PermissionMatrixPerRole	PASS	0
6	Domain.UserFactoryBuildsEachRoleAndValidates	PASS	0
7	Domain.EnumerationsRoundTripThroughText	PASS	0
8	Domain.EffectiveDeadlineAndCurrentVersion	PASS	0
9	Common.TimeConversionAndTrim	PASS	0
10	Data.ForeignKeysAreEnforced	PASS	2
11	Data.AuditLogIsAppendOnly	PASS	3
12	Data.ConstraintsRejectInvalidRows	PASS	3
13	Data.QueriesAreParameterisedAgainstInjection	PASS	2
14	Data.UnitOfWorkRollsBackAndCommitsAndNests	PASS	1
15	Data.RepositoriesRoundTripEntities	PASS	2
16	Data.ExtensionIsUpsertedPerStudentAndAssignment	PASS	2
17	Data.AuditFilterByDepartmentUserAndTime	PASS	4
18	Data.BackupCreatesAnOpenableCopy	PASS	4
19	Auth.LoginSucceedsAndIsCaseInsensitiveOnUsername	PASS	2
20	Auth.WrongPasswordAndUnknownUserGiveTheSameMessage	PASS	3
21	Auth.DeactivatedAccountCannotLogIn	PASS	3
22	Auth.AccountLocksAfterFiveFailuresAndUnlocksAfterFiveMinutes	PASS	4
23	Auth.SuccessfulLoginResetsTheFailureCounter	PASS	3
24	Auth.PasswordsAreStoredAsSaltedHashesOnly	PASS	1
25	Auth.ChangePasswordChecksOldPasswordAndRules	PASS	2
26	Admin.CreateUserValidatesEveryField	PASS	1
27	Admin.HeadOfDepartmentRules	PASS	2
28	Admin.DepartmentsCoursesEnrolmentsAndAssignments	PASS	1
29	AssignmentSvc.CreateAssignmentValidation	PASS	1
30	AssignmentSvc.ExtensionRules	PASS	2
31	AssignmentSvc.ListingsRespectRoles	PASS	2
32	Submission.OnTimeFirstSubmission	PASS	2
33	Submission.DeadlineBoundaries	PASS	14
34	Submission.ResubmissionBeforeDeadlineKeepsAllVersions	PASS	2

#	Test (suite.test)	Result	Time (ms)
35	Submission.CannotResubmitAfterDeadlineOrAfterLateOrAfterMarking	PASS	7
36	Submission.OnlyEnrolledStudentsMaySubmit	PASS	2
37	Submission.InvalidFilesAreRejectedAndStoreNothing	PASS	13
38	Submission.ExtensionMovesTheEffectiveDeadline	PASS	2
39	Submission.DatabaseFailureLeavesNoPartialSubmission	PASS	2
40	Marking.CannotMarkBeforeTheEffectiveDeadline	PASS	2
41	Marking.OnTimeMarkHasNoPenaltyAndWritesAnAuditEntry	PASS	2
42	Marking.LatePenaltyIsApplied	PASS	10
43	Marking.ExtensionRemovesThePenalty	PASS	2
44	Marking.RejectsInvalidMarksFeedbackAndWrongPeople	PASS	2
45	Marking.RemarkRequiresReasonAndIsAudited	PASS	2
46	Marking.FailedAuditRollsBackTheMark	PASS	2
47	Moderation.ReviewIsLimitedToOwnDepartment	PASS	2
48	Moderation.StudentsNeverSeeUnreleasedMarks	PASS	2
49	Moderation.AdjustMarkBeforeApprovalIsAuditedAndValidated	PASS	2
50	Moderation.ApproveAndReleaseBatchesSkipWrongStates	PASS	2
51	Moderation.ReleaseRequiresApprovalFirst	PASS	2
52	Moderation.BatchIsAllOrNothing	PASS	2
53	Moderation.CorrectionOfAReleasedMarkIsAuditedAndVisible	PASS	2
54	Moderation.AuditLogAccessIsRoleBased	PASS	2
55	Robustness.AtLeastFiftyInvalidInputsAreRejectedCleanly	PASS	2
56	Robustness.DatabaseFailureBecomesAnErrorNotACrash	PASS	1
57	Performance.LoginListingAndSavingMeetTheTargetsWithEightHundredStudents	PASS	216
58	Console.FailedLoginShowsAPlainMessageAndQuitWorks	PASS	1
59	Console.StudentSubmitsWork	PASS	2
60	Console.LateSubmissionWarnsAboutPenalty	PASS	2
61	Console.LecturerMarksAndHodModeratesAndReleases	PASS	2
62	Console.StudentSeesNothingBeforeRelease	PASS	2
63	Console.AdministratorCreatesAnAccount	PASS	1
64	Console.InvalidInputIsExplainedAndDoesNotCrash	PASS	2
65	Console.EndOfInputInsideASessionEndsCleanly	PASS	2
66	Console.ChangePasswordAndAuditLogScreens	PASS	2
67	UC18.Main_OnTimeSubmission	PASS	2
68	UC18.A1_ResubmissionBeforeDeadline	PASS	2
69	UC18.A2_LateSubmissionIsAcceptedAndFlagged	PASS	2
70	UC18.A2_SubmissionAfterSeventyTwoHoursIsStillAccepted	PASS	2
71	UC18.E1_StudentNotEnrolled	PASS	2

#	Test (suite.test)	Result	Time (ms)
72	UC18.E2_InvalidFile	PASS	1
73	UC18.E3_ResubmissionOfLateWorkRefused	PASS	2
74	UC18.E4_SubmissionAfterMarkingRefused	PASS	2
75	UC18.E5_StorageErrorLeavesNoPartialVersion	PASS	2
76	UC10.Main_MarkAfterDeadlineHiddenFromStudent	PASS	2
77	UC10.Main_LatePenaltyShownAndApplied	PASS	2
78	UC10.A1_RemarkWithReason	PASS	2
79	UC10.E1_MarkOutsideRange	PASS	2
80	UC10.E2_FeedbackMissing	PASS	2
81	UC10.E3_DeadlineNotYetPassed	PASS	2
82	UC10.E4_ModeratedOrReleasedMarkCannotBeChangedByLecturer	PASS	2
83	UC10.E5_DatabaseErrorLeavesNothingSaved	PASS	3
84	UC14.Main_ApproveThenRelease	PASS	2
85	UC14.A1_AdjustAMark	PASS	2
86	UC14.A2_PartialRelease	PASS	2
87	UC14.A3_CorrectAReleasedMark	PASS	2
88	UC14.E1_UnmarkedSubmissionIsSkipped	PASS	2
89	UC14.E2_CourseOutsideDepartment	PASS	2
90	UC14.E3_ReasonMissingOrMarkOutOfRange	PASS	2
91	UC14.E4_BatchRolledBackOnDatabaseError	PASS	2
92	AccessControl.EachRoleIsLimitedToItsOwnFunctions	PASS	1
93	AccessControl.DeactivatedUserCannotPerformActions	PASS	1
94	CR1.ClassExtensionGivesEveryEnrolledStudentTheNewDeadline	PASS	2
95	CR1.NeverShortensALaterIndividualExtension	PASS	1
96	CR1.ExtendsAnEarlierIndividualExtension	PASS	1
97	CR1.ClassExtensionRemovesThePenaltyWhenMarking	PASS	2
98	CR1.ValidationAndAccessRules	PASS	1
99	CR1.ClassExtensionIsAllOrNothing	PASS	1
100	CR1.StudentsSeeTheirExtensionOnTheAssignmentList	PASS	2
101	CR1.ConsoleLecturerGrantsAClassExtensionAndStudentsSeeIt	PASS	2
102	CR1.ConsoleShowsNoExtensionTextWhenThereIsNone	PASS	1
103	CR1.ConsoleReportsStudentsWhoKeepALaterExtension	PASS	2
104	Concurrency.FiveSessionsWriteToOneDatabaseFileWithoutCorruption	PASS	168

Appendix C: User manuals (version 1.1)

Start the program (your technician will tell you how). Type your **username** and press Enter, then your **password** (the letters do not show while you type). To leave the program, press Enter with an empty username.

Every menu has numbers. Type the number and press Enter. **0** logs you out. If you type something the program does not understand, it tells you what to do and you can try again; nothing is lost.

Dates and times are typed as `YYYY-MM-DD HH:MM`, for example `2026-11-20 17:00`.

All users: changing your password

Choose **Change my password**. Type your current password, then a new one. A new password needs at least 8 characters with a letter and a digit. After 5 wrong passwords in a row an account is locked for 5 minutes.

Student

Menu: 1 My courses and assignments, 2 Submit work, 3 See a released mark and feedback, 4 Change my password.

- **See your work to do.** Choose 1. You see each assignment, its maximum mark, its deadline and your status (NOT SUBMITTED, SUBMITTED, LATE, MARKED, MODERATED or RELEASED). If your lecturer has given you more time, the line also shows **(extended to ...)** with your new deadline.
- **Submit work.** Choose 2, then the course number, then the assignment number, then type where the file is (for example `C:\work\essay.pdf` or `/home/sam/essay.pdf`). A file must not be empty and must be 5 MB or smaller. The program shows the version number and the time it received your work.
- **Fix a mistake.** Submit again before the deadline. The newest version is the one that is marked; older versions are kept.
- **Late work.** You can still submit after the deadline. The program tells you how many days late you are. Each started 24 hours costs 10% of the assignment's maximum mark, for up to 3 days; after that the mark is zero. Late work cannot be replaced.
- **See your mark.** Choose 3. A mark and the written feedback appear only after your head of department has released them.

Lecturer

Menu: 1 My courses and assignments, 2 Create an assignment, 3 Grant an extension to a student, 4 View submissions of an assignment, 5 Mark a submission, 6 Re-mark a submission, 7 Grant an extension to a whole class, 8 Change my password.

- **Create an assignment.** Choose 2: course, title, description, deadline and maximum mark (1 to 1000). The deadline must be in the future.
- **Grant an extension.** Choose 3: course, assignment, the student, the new deadline (later than the assignment deadline) and a reason of at least 5 characters. Late penalties then count from the new deadline for that student.
- **Extend for the whole class (new in version 1.1).** Choose 7: course, assignment, the new deadline (later than the assignment deadline) and a reason. Every student enrolled in the course gets the new deadline. A student who already has a later extension keeps it; the program tells you how many students were changed. Students who join the course later are not included, so repeat the action for them.
- **Mark.** Marking opens after the student's deadline. Choose 4 to see the submission numbers, then 5. Type the mark (0 to the maximum, before any late penalty) and written feedback. The program shows the penalty and the final mark. The student cannot see the mark yet: the head of department must release it first.

- **Change a mark.** Choose 6 with a reason. This works only until the head of department has approved the mark. The old and new values and your reason are recorded.

Head of department

Menu: 1 Courses and assignments in my department, 2 Review marks of an assignment, 3 Adjust a mark before approval, 4 Approve marks, 5 Release approved marks, 6 Correct a released mark, 7 View the audit log, 8 Change my password.

- **Review.** Choose 2 to see every submission for an assignment with the raw mark, late days and final mark.
- **Adjust.** Choose 3 with the submission number, the new final mark and a reason (recorded).
- **Approve and release.** Choose 4 and type a submission number or ALL to approve all marked work. Then choose 5 the same way to release. Students see their marks only after release. Work that is not in the right state is skipped and counted.
- **Correct after release.** Choose 6 with a reason. The student sees the corrected mark.
- **Audit log.** Choose 7 to see who changed which mark, when, the old and new value and why (your department only).

Administrator

Menu: 1 Create a user account, 2 List user accounts, 3 Deactivate or activate an account, 4 Create a department, 5 Create a course, 6 Enrol a student in a course, 7 Assign a lecturer to a course, 8 List departments and courses, 9 View the audit log, 10 Back up the database, 11 Change my password.

- **Create accounts.** Choose 1, pick the role, then username (3 to 32 letters, digits, underscore or full stop), full name and a first password. A head of department needs a department; each department has one head.
- **Courses.** Create the department first (4), then the course (5), then enrol students (6) and assign lecturers (7) using the account numbers from option 2.
- **Back up.** Choose 10 and type a file name. Do this every week.

Appendix D: Updated risk register

Likelihood (L) and impact (I) are scored 1 to 5; exposure = L x I. “Initial” is the Phase 1 score; “Final” is the residual score at completion.

Table D1: Risk register at completion

ID	Risk	Type	Initial L x I	Mitigation	Owner	Final L x I	Status and evidence
R1	Schedule overruns (7 weeks fixed)	Project	4 x 5 = 20	MoSCoW; Must items first; weekly check	Developer	2 x 3 = 6	Closed. All Must and Should requirements built; FR-23 (Could) descoped
R2	Stakeholder conflicts unresolved	People	4 x 4 = 16	Written resolutions baselined at Gate 1	Developer	1 x 3 = 3	Closed. Eleven resolutions in the SRS; assumptions listed for confirmation
R3	Skill gaps in C++, SQLite, CI	Technical	3 x 4 = 12	Small interfaces; early tests	Developer	2 x 3 = 6	Mitigated. 104 tests, 98.6% coverage of business logic
R4	Database lost or corrupted	Technical	2 x 5 = 10	Transactions; backup; restore guide	IT officer	2 x 3 = 6	Mitigated. Backup function (FR-22), all-or-nothing tests, install guide. Needs a rehearsed restore on site
R5	Stakeholders unavailable for review	People	3 x 3 = 9	Early bookings; supervisor as proxy	Developer	3 x 2 = 6	Open. NFR-08 usability test with real lecturers still to do
R6	Scope creep	Project	3 x 3 = 9	Change control; out-of-scope list	Developer	1 x 2 = 2	Closed. Only CR1 accepted, through impact analysis
R7	Hashing library does not build on old machines	Technical	2 x 4 = 8	Early CI build; interface	Developer	2 x 3 = 6	Open. Built on Ubuntu 24.04 (libsodium 1.0.18) only; try the school’s computers before go-live
R8	Slow with 800 students	Technical	2 x 3 = 6	Indexes; performance test	Developer	1 x 2 = 2	Closed. Login 78 ms; listing and saving a few milliseconds
R9	New: students who join after a class extension miss it	Project	n/a	Documented; lecturer repeats the action	Developer	3 x 2 = 6	Open (accepted limitation of CR1, see `CHANGE_REQUEST_CR1.md`)
R10	New: SQLite on a network folder can lock badly	Technical	n/a	Single computer is the supported set-up; shared folder only for light use; busy timeout and test	IT officer	2 x 4 = 8	Open. Concurrency test passes on one computer, not on a real school network
R11	New: Windows 10 users cannot run the Linux build	Project	n/a	Supported through WSL2; document in install guide	Developer	3 x 3 = 9	Open. Native Windows build untested

Appendix E: Data dictionary and SQL schema

Data dictionary

Table	Column	Type	Constraints	Description
department	id	INTEGER	PK	Department identifier
	name	TEXT	NOT NULL, UNIQUE	Department name
user_account	id	INTEGER	PK	Account identifier
	username	TEXT	NOT NULL, UNIQUE, NOCASE	Login name
	full_name	TEXT	NOT NULL	Display name
	role	TEXT	NOT NULL, one of STUDENT, LECTURER, HOD, ADMIN	The account's single role (BR-9)
	password_hash	TEXT	NOT NULL	Argon2id hash with embedded salt
	department_id	INTEGER	FK department; required exactly when role is HOD	Department headed by a HoD
	is_active	INTEGER	NOT NULL, 0 or 1	Deactivated accounts cannot log in
	failed_logins	INTEGER	NOT NULL, at least 0	Consecutive failed logins (FR-21)
	locked_until	TEXT	nullable	Lock expiry, ISO 8601 UTC
course	id	INTEGER	PK	Course identifier
	code	TEXT	NOT NULL, UNIQUE	Course code
	title	TEXT	NOT NULL	Course title
	department_id	INTEGER	NOT NULL, FK department	Owning department
enrolment	student_id	INTEGER	PK, FK user_account	Enrolled student
	course_id	INTEGER	PK, FK course	Course
teaching	lecturer_id	INTEGER	PK, FK user_account	Assigned lecturer
	course_id	INTEGER	PK, FK course	Course
assignment	id	INTEGER	PK	Assignment identifier
	course_id	INTEGER	NOT NULL, FK course	Course
	title	TEXT	NOT NULL	Title
	description	TEXT	NOT NULL	Instructions
	deadline	TEXT	NOT NULL	Deadline, ISO 8601 UTC
	max_mark	INTEGER	NOT NULL, 1 to 1000	Maximum mark (FR-05)
	created_by	INTEGER	NOT NULL, FK user_account	Lecturer who created it
	created_at	TEXT	NOT NULL	Creation time
extension	id	INTEGER	PK	Extension identifier
	assignment_id	INTEGER	NOT NULL, FK assignment	Assignment
	student_id	INTEGER	NOT NULL, FK user_account	Student receiving it
	new_deadline	TEXT	NOT NULL	Extended deadline (BR-1)

Table	Column	Type	Constraints	Description
	reason	TEXT	NOT NULL, at least 5 characters	Reason (BR-10)
	granted_by	INTEGER	NOT NULL, FK user_account	Lecturer
	granted_at	TEXT	NOT NULL	Time granted
submission	id	INTEGER	PK	Submission identifier
	assignment_id	INTEGER	NOT NULL, FK assignment	Assignment
	student_id	INTEGER	NOT NULL, FK user_account	Submitting student
	state	TEXT	NOT NULL, one of SUBMITTED, LATE, MARKED, MODERATED, RELEASED	Life cycle state (BR-5)
submission_version	id	INTEGER	PK	Version identifier
	submission_id	INTEGER	NOT NULL, FK submission	Parent submission
	version_no	INTEGER	NOT NULL, at least 1	Version counter (BR-6)
	stored_path	TEXT	NOT NULL	File location in application folder
	original_name	TEXT	NOT NULL	Name the student used
	size_bytes	INTEGER	NOT NULL, 1 to 5,242,880	File size (FR-07)
	submitted_at	TEXT	NOT NULL	Submission time
mark	submission_id	INTEGER	PK, FK submission	One mark per submission
	raw_mark	REAL	NOT NULL, at least 0	Lecturer’s mark
	days_late	INTEGER	NOT NULL, at least 0	Result of BR-2 at marking time
	final_mark	REAL	NOT NULL, at least 0	Mark after penalty or HoD adjustment
	feedback	TEXT	NOT NULL, not blank	Written feedback (FR-12)
	marked_by, marked_at	INTEGER, TEXT	NOT NULL, FK user_account	Lecturer and time
	approved_by, approved_at	INTEGER, TEXT	both set or both null	HoD approval (FR-15)
	released_by, released_at	INTEGER, TEXT	both set or both null; requires approval	HoD release (FR-16)
audit_log	id	INTEGER	PK, autoincrement	Entry identifier
	submission_id	INTEGER	NOT NULL, FK submission	Submission whose mark changed
	changed_by	INTEGER	NOT NULL, FK user_account	Who (FR-13)
	changed_at	TEXT	NOT NULL	When
	action	TEXT	NOT NULL, one of INITIAL_MARK, REMARK, MODERATION_ADJUST, RELEASED_CORRECTION	Kind of change
	old_value	REAL	nullable	Previous final mark, null for the first mark
	new_value	REAL	NOT NULL	New final mark
	reason	TEXT	NOT NULL, at least 5 characters	Why (BR-10)

SQL schema

```
PRAGMA foreign_keys = ON;

CREATE TABLE department (
  id   INTEGER PRIMARY KEY,
  name TEXT NOT NULL UNIQUE
);

CREATE TABLE user_account (
  id            INTEGER PRIMARY KEY,
  username      TEXT NOT NULL UNIQUE COLLATE NOCASE,
  full_name     TEXT NOT NULL,
  role          TEXT NOT NULL CHECK (role IN ('STUDENT','LECTURER','HOD','ADMIN')),
  password_hash TEXT NOT NULL,
  department_id INTEGER REFERENCES department(id),
  is_active     INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
  failed_logins INTEGER NOT NULL DEFAULT 0 CHECK (failed_logins >= 0),
  locked_until  TEXT,
  CHECK ((role = 'HOD') = (department_id IS NOT NULL))
);
CREATE UNIQUE INDEX ux_hod_department ON user_account(department_id) WHERE role =
'HOD';

CREATE TABLE course (
  id            INTEGER PRIMARY KEY,
  code          TEXT NOT NULL UNIQUE,
  title         TEXT NOT NULL,
  department_id INTEGER NOT NULL REFERENCES department(id)
);

CREATE TABLE enrolment (
  student_id INTEGER NOT NULL REFERENCES user_account(id),
  course_id  INTEGER NOT NULL REFERENCES course(id),
  PRIMARY KEY (student_id, course_id)
);
CREATE TABLE teaching (
  lecturer_id INTEGER NOT NULL REFERENCES user_account(id),
  course_id   INTEGER NOT NULL REFERENCES course(id),
  PRIMARY KEY (lecturer_id, course_id)
);

CREATE TABLE assignment (
  id          INTEGER PRIMARY KEY,
  course_id   INTEGER NOT NULL REFERENCES course(id),
  title       TEXT NOT NULL,
  description TEXT NOT NULL,
  deadline    TEXT NOT NULL,
  max_mark    INTEGER NOT NULL CHECK (max_mark BETWEEN 1 AND 1000),
  created_by  INTEGER NOT NULL REFERENCES user_account(id),
  created_at  TEXT NOT NULL
);

CREATE TABLE extension (
  id            INTEGER PRIMARY KEY,
  assignment_id INTEGER NOT NULL REFERENCES assignment(id),
  student_id    INTEGER NOT NULL REFERENCES user_account(id),
  new_deadline  TEXT NOT NULL,
  reason        TEXT NOT NULL CHECK (length(reason) >= 5),
  granted_by    INTEGER NOT NULL REFERENCES user_account(id),
  granted_at    TEXT NOT NULL,
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission (
  id            INTEGER PRIMARY KEY,
  assignment_id INTEGER NOT NULL REFERENCES assignment(id),
```

```
  student_id    INTEGER NOT NULL REFERENCES user_account(id),
  state         TEXT NOT NULL
                CHECK (state IN
('SUBMITTED','LATE','MARKED','MODERATED','RELEASED')),
  UNIQUE (assignment_id, student_id)
);

CREATE TABLE submission_version (
  id            INTEGER PRIMARY KEY,
  submission_id INTEGER NOT NULL REFERENCES submission(id) ON DELETE CASCADE,
  version_no    INTEGER NOT NULL CHECK (version_no >= 1),
  stored_path   TEXT NOT NULL,
  original_name TEXT NOT NULL,
  size_bytes    INTEGER NOT NULL CHECK (size_bytes BETWEEN 1 AND 5242880),
  submitted_at  TEXT NOT NULL,
  UNIQUE (submission_id, version_no)
);

CREATE TABLE mark (
  submission_id INTEGER PRIMARY KEY REFERENCES submission(id),
  raw_mark      REAL NOT NULL CHECK (raw_mark >= 0),
  days_late     INTEGER NOT NULL CHECK (days_late >= 0),
  final_mark    REAL NOT NULL CHECK (final_mark >= 0),
  feedback      TEXT NOT NULL CHECK (length(trim(feedback)) > 0),
  marked_by     INTEGER NOT NULL REFERENCES user_account(id),
  marked_at     TEXT NOT NULL,
  approved_by   INTEGER REFERENCES user_account(id),
  approved_at   TEXT,
  released_by   INTEGER REFERENCES user_account(id),
  released_at   TEXT,
  CHECK ((approved_by IS NULL) = (approved_at IS NULL)),
  CHECK ((released_by IS NULL) = (released_at IS NULL)),
  CHECK (released_at IS NULL OR approved_at IS NOT NULL)
);

CREATE TABLE audit_log (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  submission_id INTEGER NOT NULL REFERENCES submission(id),
  changed_by    INTEGER NOT NULL REFERENCES user_account(id),
  changed_at    TEXT NOT NULL,
  action        TEXT NOT NULL CHECK (action IN

('INITIAL_MARK','REMARK','MODERATION_ADJUST','RELEASED_CORRECTION')),
  old_value     REAL,
  new_value     REAL NOT NULL,
  reason        TEXT NOT NULL CHECK (length(reason) >= 5)
);
CREATE TRIGGER audit_no_update BEFORE UPDATE ON audit_log
  BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;
CREATE TRIGGER audit_no_delete BEFORE DELETE ON audit_log
  BEGIN SELECT RAISE(ABORT, 'audit log is append-only'); END;

CREATE INDEX ix_enrolment_course   ON enrolment(course_id);
CREATE INDEX ix_teaching_course    ON teaching(course_id);
CREATE INDEX ix_assignment_course  ON assignment(course_id);
CREATE INDEX ix_submission_assign  ON submission(assignment_id);
CREATE INDEX ix_submission_student ON submission(student_id);
CREATE INDEX ix_audit_submission   ON audit_log(submission_id);
```

Rules that a CHECK constraint cannot express across tables, such as “the student is enrolled in the assignment’s course” and “a final mark does not exceed the assignment’s maximum”, are enforced in the services and covered by tests. The append-only triggers support NFR-07, and the indexes support the 1 second target of NFR-02 and the 2 second login of NFR-01 (the unique index on `username`).